Priroda Unofficial Manual

Program by Dimitri Laikov

Oct 18, 2026

PART I. INTRODUCTION

1 About Priroda **2**
- 1.1 Main features . . . 2
- 1.2 Capability matrix . . . 3
- 1.3 What Priroda does not do . . . 4
- 1.4 Version history . . . 5
- 1.5 How to cite . . . 6

2 Installing and running **7**
- 2.1 Obtaining Priroda . . . 7
- 2.2 Executables . . . 8
- 2.3 Running a calculation . . . 9
- 2.4 Memory, disk and scratch: `$system` . . . 11
- 2.5 Parallel performance . . . 13
- 2.6 Batch systems . . . 13
- 2.7 Input file requirements . . . 15
- 2.8 Basis-set files . . . 16

3 Getting started **17**
- 3.1 Before you start . . . 17
- 3.2 A single-point energy . . . 17
- 3.3 Optimizing a geometry and confirming a minimum . . . 20
- 3.4 Reading the output . . . 25
- 3.5 Where next . . . 26

4 Input syntax **27**
- 4.1 Groups . . . 27
- 4.2 Keywords . . . 28
- 4.3 Values . . . 29
- 4.4 The `$molecule` group . . . 29
- 4.5 Text outside groups . . . 30
- 4.6 Appending results of earlier runs . . . 31
- 4.7 Common mistakes . . . 32

5 Input reference **34**
- 5.1 `$system` . . . 34
- 5.2 `$control` . . . 36
- 5.3 `$molecule` . . . 43
- 5.4 `$guess` . . . 45
- 5.5 `$scf` . . . 47
- 5.6 `$dft` . . . 49

5.7 $grid . . . 50
5.8 $integral . . . 51
5.9 $atoms . . . 52
5.10 $optimize . . . 52
5.11 $d2edr2 . . . 58
5.12 $thermo . . . 59
5.13 $nmr . . . 60
5.14 $Energy . . . 61
5.15 $dynamic . . . 63
5.16 $uniq . . . 65

6 Solving common problems **66**
6.1 The input is not read, or is read wrongly . . . 66
6.2 SCF does not converge or diverges . . . 68
6.3 Geometry optimization problems . . . 72
6.4 Wrong or suspicious results . . . 74
6.5 Memory, disk and scratch errors . . . 76
6.6 Parallel and MPI problems . . . 77

7 Keywords in the program **80**
7.1 Groups for everyday work . . . 80
7.2 Groups of little-documented tasks . . . 81
7.3 Task names . . . 81
7.4 Theory names . . . 82

8 Sources **83**
8.1 Test runs . . . 83
8.2 Program files from the developer . . . 83
8.3 Unofficial keyword lists . . . 83
8.4 User-written material . . . 84
8.5 Published papers . . . 84

Index **85**

Important

This manual is not an official Priroda document. It was not written or reviewed by the developer. It is compiled from the files on the official site, unofficial keyword lists and wiki pages, test runs, and user reports from forums and user pages; *Sources* lists them all. When this manual and the program disagree, the program is right.

CHAPTER

ONE

ABOUT PRIRODA

Priroda (Russian for "nature") is a quantum-chemistry program for molecules written by Dimitri N. Laikov. It began as a fast program for GGA density-functional calculations and grew to cover hybrid DFT, RI-MP2, coupled-cluster methods, TDDFT, a scalar-relativistic Hamiltonian, NMR, and Laikov's own semiempirical model.

The program is closed-source and has no official manual. The official site offers executables, basis-set files and example inputs, but no prose documentation. The other documents in circulation are unofficial: keyword lists such as the English "Input Summary" (`input.txt`), and a keyword page and beginner FAQ on a computing-centre wiki. This manual collects those texts, the example files on the official site, test runs of version 25, and twenty years of user experience from Russian-language forums. It is not an official document: the developer has not written or reviewed it. *Sources* lists what it is based on. To download the program, see *Obtaining Priroda*.

1.1 Main features

Density-functional theory
: GGA DFT in which the Coulomb and exchange–correlation terms are evaluated with an auxiliary (fitting) basis. This is the source of Priroda's speed. Every orbital basis comes with a matched fitting basis. GGA DFT has analytic second derivatives.

  Hybrid functionals with exact exchange evaluated by the resolution of the identity, under *theory=ridft*.

Correlated methods
: RI-MP2, and conventional MP2–MP4, CCD, CCSD and CCSD(T); version 25 runs energies with all of them. Gradients exist for MP2, CCSD and CCSD(T).

Excited states
: GGA TDDFT with excited-state gradients, so excited-state geometries can be optimized.

Relativistic calculations
: A scalar-relativistic Hamiltonian (*four=1*) with all-electron basis sets and a finite-nucleus model. The user FAQ of one computing centre names this, together with basis sets for the whole periodic table, as what sets Priroda apart. Effective core potentials (SBK) are also available.

Geometry and dynamics
: Minimization in redundant internal coordinates, transition-state search, IRC, relaxed scans, Hessians with vibrational analysis and thermochemistry, and molecular dynamics with a thermostat.

Properties
: NMR shielding tensors and chemical shifts (also at points in space, for NICS), Hirshfeld and Mulliken charges, bond orders, ESR parameters, critical points of the electron density, Raman intensities, and orbitals and normal modes for Molden.

Semiempirical model
Laikov's parametrized "QM" model (*theory=qm_n3*), used for systems of thousands of atoms.

Parallel execution
MPI parallelism. Users report no gain from more than one node. Within one node the reports differ: one user saw little gain beyond 6–8 cores, another a good speed-up up to 24 cores. See *Installing and running*.

Experienced users report that Priroda is markedly faster than general-purpose programs for routine GGA work. One 2018 comparison found it about three times faster per optimization step than ORCA 4.0 (PBE/cc-pVDZ, one core).

1.2 Capability matrix

The table shows which `task=` values are known to work with each `theory=` value. It is assembled from example inputs, user reports and the test runs on version 25, not from a statement by the developer.

- ✓: shown to work in an example input, a user report or a test run.
- –: not available, according to the sources.

Task	`dft`	`ridft`	`tddft`	`rimp2`	`ccsd`	`ccsd4t`	`qm_n3`
`energy`	✓	✓	✓	✓	✓	✓	✓
`gradient`	✓	✓	✓	✓	✓ (1)	✓ (1)	✓
`hessian`	✓ (2)	✓ (3)	✓ (3, 8)	✓ (3)	✓ (3)	✓ (3)	✓ (3)
`optimize`	✓	✓	✓ (4)	✓	✓ (1)	✓ (1)	✓
TS (`saddle=1`)	✓	✓	✓	✓	✓	✓	✓
`irc`	✓	✓	✓	✓	✓	✓	✓
`scan`	✓	✓	✓ (9)	✓	✓	✓	✓
`nmr`	✓ (5)	–	–	–	–	–	–
TDDFT excited states	–	– (6)	✓	–	– (7)	– (7)	–
MD (`thermostat`)	✓	✓	✓	✓	✓	✓	✓

Notes to the table:

1. A CCSD optimization is shown in a user input, and a user reports CCSD(T) optimizations and Hessians. A 2011 bug fix that names the CCSD(T) gradient (Laikov, in an e-mail relayed on the qchem.ru forum) also implies CCSD(T) gradients. Version 25 tests confirmed gradients and optimizations with both `ccsd` and `ccsd4t`; the `ccsd4t` energies include the (T) correction.
2. Analytic second derivatives.
3. Computed by finite differences of the gradient (`NUMERICAL COMPUTATION OF SECOND DERIVATIVES`), which takes many gradient calculations. Shown for `ridft`, `rimp2`, `tddft`, `qm_n3`, `ccsd` and `ccsd4t` in version 25 tests.
4. Excited-state optimization works, but Priroda has no root following, so the state can switch during the optimization.
5. NMR works only with `theory=dft` and a non-hybrid functional; with `ridft`, version 25 silently computes only the energy. Relativistic NMR is not implemented (Laikov, relayed by a user), and users report that NMR runs with `four=1` hang.
6. There is no hybrid TDDFT. See *What Priroda does not do*.
7. Excited states at the coupled-cluster level use a different `theory=` value, `ccsdsd`, for closed-shell (RHF) molecules, with the state chosen by `state=`. In a version 25 test on water it gave the first excited state

0.338 hartree (9.2 eV) above the CCSD ground state, against 0.370 hartree for CIS in the same basis. For an open-shell molecule it silently computed only the UHF energy. It is not in `input.txt`; only the v5 keyword page and the program itself list it.

8. The Hessian of the excited state given by `state=` (version 25, water, `state=1,3`): numerical, with two imaginary modes. The geometry was the ground-state test geometry, not optimized for the excited state, so these numbers only show negative curvature there.

9. A TDDFT scan of the NH3 umbrella angle in its first excited state worked in version 25 when all three N–H bonds were fixed. With only one fixed, the excited state broke another N–H bond (it is dissociative), the excitation energy fell from 7.3 to 0.4 eV over 46 steps, and the run then failed: the excited-state equations diverged (`g= -nan`) and it stopped with `0: write td.XK(...)?`. TDDFT cannot follow an excited state that comes down to the ground state; watch the excitation energy (`DE =`) in excited-state optimizations.

The version 25 tests behind the TS, IRC, scan and MD rows used tiny molecules (NH3, water) and short runs, so they show that the combination runs and ends normally, not how well it behaves on real problems.

The scalar-relativistic option `four=1` has been used with `dft`, `ridft` and `rimp2`. The full list of `theory=` values, including the undocumented ones, is in *$control theory*.

Warning

Hybrid functionals work only with `theory=ridft`. With `theory=dft`, older versions silently drop the exact exchange, and newer versions, including version 25, stop right after printing the basis sets with no error message in the output. With `theory=tddft`, version 25 still runs a hybrid without its exact exchange and gives wrong energies with no warning. See *$control theory*.

1.3 What Priroda does not do

Check this list before planning a project around Priroda.

Limited use of point-group symmetry
: Symmetry is used only to symmetrize a structure and keep it symmetric; no source reports a speed-up from it. Older versions always run in C1. A symmetric starting structure stays symmetric during optimization, so a Hessian is needed to show whether it is a minimum. Newer outputs (v13 onwards) print a detected point group, and the official 2023–2025 example inputs use `sym=1` in `$molecule` and `$optimize`. These options are not described in any written source. In version 25 tests, `$molecule sym=1,0.01` symmetrized the structure and `$optimize sym=1` kept the symmetry during the optimization; each option alone does only half of this. `$optimize sym=1` also switches the optimizer to Cartesian coordinates. See *$molecule sym* and *$optimize symmetry*.

No continuum solvation
: Users reported no continuum solvent model in any version up to 2018, and the version 25 executable contains no group for one. Solvent can only be included as explicit molecules.

No D3 or other empirical dispersion correction
: The developer has declined to add one (relayed from correspondence). Users add Grimme's D3 a posteriori with the external `dftd3` program. The 2023–2025 functionals `LPBEVV` and `LPBEVVV` may include dispersion, but this is inferred from their test systems and not documented.

No periodic systems
: Priroda handles molecules and clusters only.

No dummy atoms, so no counterpoise (BSSE) correction
: Ghost centres cannot be placed in `$molecule`. For NICS, give the probe points in *$nmr points* instead.

No hybrid TDDFT
: TDDFT is GGA only. With a hybrid functional, version 25 drops the exact exchange without a warning (see the warning above this section).

Other gaps reported by users:

- no spin–orbit coupling, so no phosphorescence calculations;
- no multireference methods (CASSCF, MRCI, CASPT2) that users could get working;
- no freezing of atoms in Cartesian space; only internal coordinates can be fixed (*$optimize fix*);
- no general job chaining in one input: `task=optimize+hessian` is the only combined task, and two `task=` lines run only the first.

1.4 Version history

Since version 6, Priroda numbers its versions with integers; versions 2 and 4 printed decimal sub-versions (2.10, 4.07). The output banner shows the build date, for example `Priroda 13 (2013.01.10)`. Energies differ slightly between versions, so keep every calculation you compare on one version.

Version	Build dates seen	Notes
2	2002.01.14 (v2.10)	Banner `program Priroda version 2.10`. Had `task=optimize+hessian`.
4	2004.09.14 (v4.07)	The finite-nucleus model changed between versions 3 and 4, so pre-v4 relativistic basis sets must not be used with v4 and later.
5	—	Version described by the keyword reference on the KNC wiki. Introduced the systematic L basis sets. Hybrid RI-DFT was implemented in November 2004.
6	2006.08.20	Long the most widely used version: 32- and 64-bit Linux builds with MPI, and a Windows build (`p6.exe`). `optimize+hessian` was removed. Parallel Hessians use at most 2 cores.
7	—	Sent as object files for linking against a local MPI library.
8	2008.09.05	Built on request for a Myrinet cluster.
9	— (forum posts 2009–2011)	`optimize+hessian` returned. In mid-2011 Laikov fixed a long-standing bug that in rare cases gave wrong gradients for MP2, MP4, CCSD, CCSD(T), CIS(2) and EOM-CCSD (from his e-mail, relayed on the qchem.ru forum).
10	2011.04.14	Linux build `p` with a bundled `mpiexec`.
11	—	Contains the semiempirical QM model.
13	2013.01.10, 2013.01.21, 2013.09.13	Builds behind the example outputs on the official site.
14	2014.01.01	Input geometry echoed with a < prefix (<`$molecule`). 8060-atom QM-model run on 32 processes.
15	2015.09.14	Scalar-relativistic example (Pt complex).
16, 17	— (forum posts 2016–2018)	The Windows build runs in parallel from version 16. Version 17 runs Hessians on 4 or more cores. Around 2016 the developer removed limits on SCF memory allocation and on the system size for Hessians.
19	2019.03.06	Banner of `p_190306.exe`, the parallel Windows build on the official site.
20	2021.03.18	Banner of `p1.exe`, the serial Windows build on the official site.
23	2023.06.22	Newest numbered version reported by users (2024). New functionals `LPBEVV` and `LPBEVVV`.
25	2025.02.28	Banner `Priroda 25 (2025.02.28)`. The version used for the test runs behind this manual. Its date matches the `p4_d` build on the official site; `p_l` is dated 2025-04-14. One 2025 example input uses `theory=DFT_n4` (with `LPBEVV`).

Versions 12, 18, 21, 22 and 24 do not appear in any source; version 3 is mentioned only for the change of the nucleus model. The Russian file `Dft_ansi.txt` describes Priroda's predecessor, whose input had the groups `$control`, `$coordinates`, `$basis`, `$auxiliary`, `$guess`, `$SCF`, `$search`, `$force` and `$hessian`. Version 25 reads none of `$coordinates`, `$auxiliary`, `$search`, `$force` and `$hessian` as an input group; the geometry now goes in `$molecule`, and optimization settings in `$optimize`. (`$AUXILIARY` survives as a section of the old-style basis files `3z.bas` and `sbk.bas` on the official site.)

1.5 How to cite

Priroda has no single program paper for the current version. Users cite the papers below. The first five are the citation list of the KNC wiki FAQ; the first three are also given in the forums.

Program:

- D. N. Laikov, Yu. A. Ustynyuk. PRIRODA-04: a quantum-chemical program suite. New possibilities in the study of molecular systems with the application of parallel computing. *Russ. Chem. Bull., Int. Ed.* **54** (2005) 820–826. Russian original: *Izv. Akad. Nauk, Ser. Khim.* 2005, No. 3, 804–810.

Density-fitting DFT (cite for any DFT calculation):

- D. N. Laikov. Fast evaluation of density functional exchange-correlation terms using the expansion of the electron density in auxiliary basis sets. *Chem. Phys. Lett.* **281** (1997) 151–156.

L-family basis sets (`L1`, `L2`, `L11`, ...):

- D. N. Laikov. A new class of atomic basis functions for accurate electronic structure calculations of molecules. *Chem. Phys. Lett.* **416** (2005) 116–120.

The `3z` basis set, which has no journal paper:

- D. N. Laikov. "Development of an economical approach to the density functional calculation of molecules and its application to complex chemical problems". PhD dissertation, Moscow State University, 2000 (in Russian).

Scalar-relativistic DFT (`four=1`):

- D. N. Laikov. An implementation of the scalar relativistic density functional theory for molecular calculations with Gaussian basis sets. DFT2000 Conference, Menton, France, 2000.

Semiempirical QM model (`theory=qm_n*`):

- D. N. Laikov. A new parametrizable model of molecular electronic structure. *J. Chem. Phys.* **135** (2011) 134120, doi:10.1063/1.3646498.

For version 6, users cited the program itself as "D. N. Laikov, PRIRODA, Electronic Structure Code, Version 6, 2006". A 2014 forum suggestion adds Laikov's home page, http://rad.chem.msu.ru/~laikov/, as the reference for the current version.

Also cite the original papers for the functional, and for the Hamiltonian and nuclear model if you use `four=1`. *$dft functional*, *$control four* and *$control nucleus* give them.

CHAPTER

TWO

INSTALLING AND RUNNING

Priroda has no installer. You copy an executable, the basis-set files you need and, for parallel runs, the MPI launcher into a directory, write an input file, and start the program with the input and output file names as arguments.

2.1 Obtaining Priroda

Priroda is free of charge. The executables, the basis-set files and the helper scripts can be downloaded from the official site, https://rad.chem.msu.ru/~laikov/p_w/. Only parallel runs on Windows need more: an MPI runtime, which is not on the site (see *Parallel runs on Windows*). Put the executable and `basis.in` in a directory of your own. The main Linux executable, `p`, is probably statically linked (see *Executables*). The page is a bare file index with no documentation, license text or contact information. It holds:

- Linux x86-64 executables (`p`, `p4`, `p_d`, `p4_d`), builds for particular clusters (`p_l`, `p_mvs1p1`, `p_mvs10p`, `p_mvs10q`), an ARM build (`p_arm`), a bundled `mpiexec`, and the molecule viewer `xm`;
- Windows (Cygwin, x86-64) executables in `exe64/`;
- example inputs and outputs;
- basis-set and parameter files (`basis.in`, `basis4.in`, `b2*.in`, `b4s*.in`, `3z.bas`, `sbk.bas`, `qm.in`);
- small shell scripts: `ge` (append the `$Energy` block from an output to the input), `gm` (replace `$molecule` with the optimized geometry) and `grx` (join two IRC halves).

Which executable to choose is described under *Executables* below.

Users have also obtained the program by writing to the developer, Dimitri Laikov, by e-mail. Users reported the address `dimitri_laikov@mail.ru` from 2010 to 2018; earlier it was `laikov@physto.se`. Users note that he does not always answer quickly.

Note

The legal status is not stated on the official site. A 2009 presentation relayed on a forum said that Priroda is registered with Rospatent and distributed through the company OOO "Dimonta". A user reported in 2018 that the developer had said, a couple of years earlier, that there was no licence agreement. Users customarily ask the developer's permission before using the program.

The official site has hosted builds with known defects. In 2016 it carried a note that the `w64/*.exe` builds (an attempt at 64-bit Cygwin) had a DFT/ECP bug. If a build behaves strangely, ask the developer whether it is the right one.

2.2 Executables

2.2.1 Linux

The official site lists these Linux files. Dates are from the official file index (checked October 2026).

File	Date	Description
`p`	2023-06-25	Main x86-64 executable, probably statically linked.
`p4`	2023-06-25	Build "for FMA/AVX2" processors.
`p_d`	2021-11-12	Dynamically linked build.
`p4_d`	2025-02-28	Dynamically linked FMA/AVX2 build.
`p_l`	2025-04-14	Build for a particular cluster, started through a cluster-side Intel MPI wrapper.
`p_mvs1p1`, `p_mvs10p`, `p_mvs10q`	2019–2023	Builds for particular clusters.
`p_arm`, `mpiexec_arm`	2015-09-23	ARM build and its MPI launcher.
`mpiexec`	2012-09-09	MPI launcher shipped with `p`.
`xm`	2025-04-24	A molecule viewer. It takes a Priroda input file as its argument (`xm C60.qm.in`).

That `p` is statically linked is inferred from the description of `p_d` as "dynamically linked". The site does not say which cluster each `p_mvs*` and `p_l` build is for.

Older distributions used other names: `p6`, `p6_32` and `p6_64` for version 6, `p9`, and `p1` for version 11 at one site. Scripts and forum posts written for those versions use these names. This manual writes `p` throughout.

Some users run the Linux build under WSL on Windows.

2.2.2 Windows

The Windows build is a Cygwin build for x86-64. It is in the `exe64/` directory of the official site:

File	Date	Description
`p1.exe`	2021-10-01	Serial executable.
`p1_190306.exe`	2019-03-12	Older serial build.
`p_190306.exe`	2019-03-12	Older parallel (MPI) build.
`xm.exe`	2019-03-08	Viewer.
`cygwin1.dll`	2021-10-01	Cygwin runtime library.

For serial runs you need only `p1.exe` and `cygwin1.dll` in the same directory. A full Cygwin installation is not required, and no administrator rights are needed. For parallel runs see *Parallel runs on Windows*.

The Windows builds are older than the Linux one: `p1.exe` prints `Priroda 20 (2021.03.18)` and `p_190306.exe` prints `Priroda 19 (2019.03.06)`. For water the serial `p1.exe` gave the same energy as Linux version 25 to all printed digits, and the same optimized geometry and frequencies. Features added after version 20 are missing from the Windows builds.

Note

In 2017 users referred to the parallel Windows build as `exe64/p.exe`. The current index has no `p.exe`; the newest parallel build there is `p_190306.exe`.

Old 32-bit Windows builds could not allocate more than about 1 GB of memory; `malloc(...) failed` appeared above that. The 64-bit `p1.exe` ran with `mem=3000` without trouble.

2.3 Running a calculation

2.3.1 Serial runs

Give the input file first and the output file second:

```
p input.in output.out
```

On Windows:

```
p1.exe input.in output.out
```

On Linux the current directory is usually not in `PATH`, so start the program as `./p` or with its full path. If the program cannot find the input or the basis file, give full paths everywhere: for the executable, the input, the output, and the basis file inside the input.

Run in a directory you own and can write to. Keep file and directory names to Latin letters, digits and simple punctuation, with no spaces or Cyrillic. This also applies to the computer name on Windows. File names are case-sensitive on Linux, although keywords are not.

The Windows `p1.exe` (version 20) did run in a folder with a space in its name, both from inside it and from outside with the paths in double quotes. Full Windows paths such as `C:\Work\mol.in` work for the input and output files.

One user of an older version saw the complaint `0:  output file 'tasK.out'?` when the output file name contained a capital letter. Version 25 writes to an output file with capital letters in its name without complaint.

If no output file is given (`p input.in`), the output goes to standard output.

2.3.2 Parallel runs on Linux

Priroda is parallelized with MPI. Start a parallel run through an MPI launcher. The official site shows the `mpiexec` that it ships next to `p`:

```
./mpiexec -np 4 ./p input.in output.out
```

The output then starts with `parallel on 4 processors` and one `node N:` line per process. A two-process test gave exactly the serial energy.

Older versions also accepted `p -np 4 input.in output.out`, which users ran with versions 6 and 10. **Version 25 does not**: it takes `-np` as the input file name, prints `input file '-np'?.` and stops. One site ran version 14 with the bundled `mpiexec` (`mpiexec -np N p input.in`, output to standard output).

Check the start of the output. A parallel run prints the number of processes and one line per process:

```
parallel on  4 processors
node  0: 'linux-ldmm'
node  1: 'linux-ldmm'
...
```

The official site gives one example for a cluster build, `p_l` under SLURM:

```
sbatch -p test -n 16 -t 6 impi ~/bin/p_l file.{in,out}
```

Here `impi` is presumably that cluster's Intel MPI wrapper script; it is not part of Priroda.

2.3.3 Parallel runs on Windows

The official site shows this command for a two-process run:

```
mpirun.exe -np 2 p.exe C60.qm.in C60.qm.w.2.out
```

The current index has no `p.exe`: use `p_190306.exe` in place of `p.exe`, here and in the commands below.

The MPI runtime is not on the site. Users assembled it from Cygwin as follows:

1. Download the 64-bit Cygwin installer (`setup-x86_64.exe` from https://cygwin.com). Install the default packages plus the `openmpi` package.
2. Copy these files from the Cygwin installation into the directory with the Priroda executable: `cygwin1.dll`, `cyggcc_s-seh-1.dll`, `cygmpi-12.dll`, `cygopen-pal-13.dll`, `cygopen-rte-12.dll`, `orterun.exe`. Take `cygwin1.dll` from the same installation, because the other DLLs work only with matching versions. After this, Cygwin itself is no longer needed.
3. Create a directory `dev` one level above the directory that holds `cygwin1.dll`, with subdirectories `shm` and `mqueue`. For Priroda in `C:\Priroda`, that means `C:\dev\shm` and `C:\dev\mqueue`. The Cygwin libraries take the parent of the directory that holds `cygwin1.dll` as the root `/`: with the DLL in `C:\Priroda`, `C:\Users` is `/Users`.
4. If the program is in a system directory such as `C:\Program Files`, run it as administrator.
5. Set `TMPDIR` and start the run from `cmd.exe`, giving the path of the Priroda executable with forward slashes:

   ```
   cd /d C:\Work
   set "TMPDIR=%TEMP%"
   C:\Priroda\orterun.exe -np 2 C:/Priroda/p_190306.exe file.in file.out
   ```

Important

`p_190306.exe` needs exactly the files `cygmpi-12.dll`, `cygopen-pal-13.dll` and `cygopen-rte-12.dll`. Current Cygwin OpenMPI packages ship `cygmpi-40.dll`, `cygopen-pal-40.dll` and `cygopen-rte-40.dll` instead. With those, or with those copied under the old names, the program and `orterun.exe` exit at once with code -1073741515 (0xC0000135, a required DLL was not found), and nothing is printed. A test in 2026 worked with the `-12`/`-13` files and `orterun.exe` from an older OpenMPI package (files dated 2016-06-21). Take `orterun.exe` from the same package as the DLLs.

That test ran on Windows 10 with a Cygwin installation on `PATH`. `p_190306.exe` then printed `parallel on 2 processors` and gave the same energy as a serial run. With a backslash path to the executable (`C:\Priroda\p_190306.exe`), `orterun.exe` stopped with `orterun:exe-not-found`. Started directly, without `orterun.exe`, `p_190306.exe` runs serially.

Problems reported with this setup:

- The recipe needs a Priroda build that supports parallel runs. Version 6 (`p6.exe`) failed with an OpenMPI `agent-not-found` error; the user was advised to use version 17 or later.
- On a computer connected by Wi-Fi that also has a wired network adapter, OpenMPI stops with `tcp_peer_send_blocking: send() to socket 12 failed: Transport endpoint is not connected (128)`. Disable the wired adapter.
- With the Ethernet cable unplugged, one user's run used only two of the cores.
- `Warning: '/dev/shm' does not exists or is not a directory.` means the Cygwin environment is missing. Create the `dev` directories (step 3), or start from a Cygwin shell.

Warning

A user reported in 2021 that this recipe no longer worked on Windows 10 with a current Cygwin, and no fix was posted. The likely cause is the newer OpenMPI DLLs described above. A PC without a Cygwin installation was not tested.

2.4 Memory, disk and scratch: `$system`

The *$system* group sets the resources. Every limit applies **per MPI process**. `disk=10` on four processes can use up to 40 GB.

```
$system memory=2048 disk=-2048 path=/scratch/user $end
```

memory
: Memory in MB. Two values set rank 0 and the other ranks separately: `mem=2048,256` gives the master process 2048 MB and every other process 256 MB. The v5 documentation says that in a parallel run most of the memory and all of the disk storage are used by rank 0. That is why a small second value works: the other ranks need less.

disk
: A positive value is scratch disk in **GB**. The program opens one scratch file per gigabyte. A negative value is scratch kept in memory, in **MB** (a RAM disk): `disk=-2048` holds 2048 MB of scratch in RAM. A second value applies to the other ranks, as for `memory`; the official example inputs typically give them `-1`.

path
: The scratch directory. A colon-separated list gives one directory per process, which pays off only when the directories are on different physical drives. For several processes and one fast disk, a single directory is enough.

2.4.1 Keeping scratch in memory

With a negative `disk`, `memory` plus —`disk`— is taken by every process, so the sum over all processes must fit into the node's physical RAM, or the run stops at once with `malloc(...) failed`. With `mem=a,b` and `disk=a,b`, rank 0 takes the first values and every other rank the second. One user's setting for a 24 GB node is `memory=9000 disk=-14000`; that fills the node with **one** process. On several processes, give the other ranks small second values or lower the limits.

Keeping the scratch in RAM is faster than a local disk; one user calls it "a bit faster". In an older cluster benchmark the gain was larger: a version 8 RI-MP2 job on 8 cores (4 nodes) took 12.4 minutes with scratch in RAM, against 72 minutes on 2 cores with scratch on disk and 86 minutes on 8 cores with direct SCF and scratch on disk.

2.4.2 Where scratch files go

The scratch directory is taken from `path=`, then from the `TMPDIR` environment variable. The files are named `tmp<number>.<suffix>`, for example `/tmp/tmp2960.0` or `/tmp/tmp9633.11`. In version 25 each process of a parallel run wrote its own file with a different number and the same suffix (`tmp5345.0000`, `tmp5346.0000`). Priroda deletes them at the end of a normal run, and in version 25 also when the job is stopped with signal 15 (SIGTERM). After a crash, delete the leftovers yourself.

When neither `path=` nor `TMPDIR` is set, version 25 puts scratch files in `/tmp` (echo `Disk:  1g '/tmp'`), as version 6 did. `input.txt` says the current working directory instead, and a 2020 user report gives the directory of the input file; neither matches version 25. `/tmp` is often small or shared, so set `path=` or `TMPDIR` for real jobs.

On Windows there is no `/tmp`. Without `path=` or `TMPDIR`, the run stops after printing `0:  opening file /tmp/tmp978.0000` (the number varies), and the output ends with `process 0 got signal 6`. This still happens with `p1.exe` (version 20). Any of these avoids it:

- `$system path=. $end` (the current directory);
- `TMPDIR` set to a Windows path: in `cmd.exe`, `set TMPDIR=%TEMP%` or `set "TMPDIR=%TEMP%"`. Do not write `set TMPDIR="%TEMP%"`: `cmd.exe` keeps the quotes in the value, Priroda takes the result as a relative path, and the run stops in the same way;
- a RAM disk (negative `disk`), which needs no scratch directory. An input with no `$system` group gets a 512 MB RAM disk and runs.

2.4.3 Disk speed

Warning

Never put scratch files on NFS or another network file system. On most clusters the home directories are on NFS; use the node-local scratch disk instead.

Priroda is sensitive to disk speed. The unofficial keyword list `input.txt` warns that for all methods except GGA DFT, performance "will be degraded terribly" if more than one process reads and writes large files on the same physical drive. In order of preference:

1. keep the scratch in RAM (negative `disk`) if it fits;
2. give each process its own physical drive with `path=a:b:c:d`;
3. put all processes on one fast local disk (SSD), accepting some loss.

If the disk is the bottleneck for RI-MP2 or RI-DFT, *$integral direct=1* recomputes the integrals instead of storing them. One user found it uses about a third of the memory but runs about ten times longer in CPU time. In a disk-bound RI-MP2 optimization another user got 20 steps overnight with it instead of 2.

Users also recommend more RAM over faster disks. Scratch on a separate SSD helps, but RAM matters more.

2.4.4 When the limits are too small

Priroda does not fall back to a slower algorithm when memory or scratch runs out. The run stops with an error such as `need ...  bytes.` (the `memory` limit is too small), `malloc(536870912) failed` (the machine cannot supply the memory) or, for scratch, `0:  out of disk space (grid_W), 8213824` (version 25; older versions printed the number with `KB`). Raise the limit in `$system`, move to a node with more resources, or choose a cheaper method (RI-MP2 instead of conventional MP2).

If you omit `$system`, version 6 printed `Memory: 128 (128) MB` and `Disk: 1 GB '/tmp'`. Version 25 prints `Memory: 256m + 512m (256m + 512m)` and no `Disk:` line. Read by the echo format, that is 256 MB of memory and a 512 MB RAM disk per process. That is enough for small molecules only, so always give `$system` for anything larger than a test.

If a run fails for lack of memory or disk, see *Memory, disk and scratch errors*.

2.5 Parallel performance

What is known about parallel performance:

- **DFT speeds up within one node; reports differ on how far.** One user found in 2016 that beyond 6–8 cores the gain is at most about 10%, even for large systems such as (C60)9. Another measured a 38-step optimization of a small molecule at 101, 63 and 37 s on single nodes with 8, 12 and 24 cores. A small PBE/L1 optimization with version 10 took 81.5, 42.5, 29.8 and 29.3 s on 1–4 cores. In a version 25 test, a numerical `ridft` Hessian of water took 4.9 s on 2 processes instead of 7.2 s. An optimization plus analytic Hessian of a 22-atom copper complex (OLYP/L2, fine grid) took 32 min on 8 processes of one node; on one process it had not finished after an hour.
- **Parallel results equal serial ones in version 25 tests.** On 2 processes, a water energy and a numerical `ridft` Hessian gave exactly the serial numbers. In version 6, results were reported to differ in the last digits between runs on different numbers of processes, and an unstable TS search once followed different paths for different `-np`.
- **Running across nodes gives no gain for DFT.** Use one node per job.
- **Memory per process does not drop** when you add processes, and in a parallel Hessian nearly all of the disk use is on rank 0.
- **RI-MP2 parallelizes better than DFT**, according to one user "at least in memory". The v5 documentation limits useful parallelism to Nproc $\leq$ min(N_atoms, N_occ) and recommends a number of processes that divides the number of occupied orbitals nearly evenly. For N_occ = 14 it rates 2, 7 or 14 processes as best, 3 or 5 as acceptable, 6 as poor and 13 as very bad.
- **Small RI jobs on many processes can hang** (reported in 2015; not tested in version 25). RI-MP2 and RI-DFT hang with no message when a process receives no work. The output then shows a line `0 blocks ... on node N` after `integral estimates`. Run small RI jobs on one or a few cores.
- **Several independent jobs often beat one parallel job.** On a 4-core machine in 2009, for jobs not limited by disk, four single-core jobs ran about 30% faster in total than the same jobs run one after another on 4 cores each.

Note

Older versions scaled worse. In version 6 a parallel Hessian could use at most 2 cores and could hang on 4, and a version 7 MP2 run crashed (signal 8) on more than about 24 cores. Use version 17 or later for parallel work.

Hangs, crashes and poor speed-up in parallel runs are covered in *Parallel and MPI problems*.

2.6 Batch systems

Priroda needs nothing special from a batch system. The job script has to:

1. request **one node** and the number of cores you will use (see *parallel performance above*);
2. point the scratch to the node's local disk or to RAM, by exporting `TMPDIR` or by `path=` in `$system`;

3. make the memory request cover `memory` + —negative `disk`— for every process;
4. start Priroda with the input and output file names.

This SLURM script was tested in version 25 on a single-node Slurm 23.11. Directory names and module setup are site-specific.

```
#!/bin/bash
#SBATCH --job-name=mol
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --time=24:00:00

PRIRODA=$HOME/priroda/p                       # full path to the executable
cd "$SLURM_SUBMIT_DIR"
export TMPDIR=/local/scratch/$SLURM_JOB_ID   # node-local disk; site-specific
mkdir -p "$TMPDIR"

env -i PATH="$PATH" HOME="$HOME" TMPDIR="$TMPDIR" \
  $HOME/priroda/mpiexec -np "$SLURM_NTASKS" "$PRIRODA" mol.in mol.out   # the mpiexec␣
↪from the official site
status=$?

rm -rf "$TMPDIR"
exit $status
```

Two lines matter more than they look:

- `env -i`. The `mpiexec` from the official site refuses to start when the environment holds 193 or more variables. It prints `Environment is too large (max is 193)` and exits, and Priroda never runs. A Slurm job easily has that many, because Slurm adds its own `SLURM_*` variables; in the test, a job inherited 210 and the unmodified script failed this way. `env -i` starts `mpiexec` with only the variables listed.
- `exit $status`. Without it the job's exit status is that of `rm`, so Slurm reports a failed run as `COMPLETED`. This is what happened in the test above.

When a job is cancelled (`scancel`) or reaches its time limit, Slurm sends signal 15 (SIGTERM). Priroda then writes `process 0 got signal 15` and then `process 0 got signal 2` to the output, deletes its own scratch files, and stops; Slurm's own log file says `CANCELLED` or `CANCELLED ... DUE TO TIME LIMIT`. The `rm` at the end of the script does not run, so the empty scratch directory stays. To remove it as well, set the clean-up as a trap before starting Priroda: replace the lines of the script from `env -i` through `exit $status` with these.

```
trap 'rm -rf "$TMPDIR"' EXIT
trap 'exit 143' TERM
env -i PATH="$PATH" HOME="$HOME" TMPDIR="$TMPDIR" \
  $HOME/priroda/mpiexec -np "$SLURM_NTASKS" "$PRIRODA" mol.in mol.out &
wait $!
```

Two jobs of one process each ran side by side on the same node without trouble, each with its own scratch directory.

Under PBS/Torque the same steps read the number of cores from `$PBS_NODEFILE`. This script was not tested; it uses `env -i` and `exit $status` for the same reasons as the SLURM one. Some PBS sites (e.g. KNC) provide a per-job local scratch directory and delete it after the job.

```
#!/bin/bash
#PBS -N mol
#PBS -l nodes=1:ppn=8,walltime=24:00:00

cd "$PBS_O_WORKDIR"
NP=$(wc -l < "$PBS_NODEFILE")
export TMPDIR=/local/scratch/$PBS_JOBID      # site-specific
mkdir -p "$TMPDIR"

env -i PATH="$PATH" HOME="$HOME" TMPDIR="$TMPDIR" \
  $HOME/priroda/mpiexec -np "$NP" $HOME/priroda/p mol.in mol.out
status=$?

rm -rf "$TMPDIR"
exit $status
```

If `/tmp` on the node is small or has a quota, a large job dies writing a scratch file, for example `0: write file /tmp/tmp9633.11 (leAX): ...`. Point the scratch at the cluster's scratch file system or at RAM.

Note

Historical: the KNC wrappers. The Kazan Scientific Centre clusters (2006–2012) provided the wrappers `pbspriroda5`, `pbspriroda5-mpi` and `pbspriroda6`, called as `pbspriroda6 file_name -time hh:mm:ss -queue queue -mem MB -disk size -procs 1:ppn=N`. The wrappers wrote the `$system` group themselves from `-mem` and `-disk`, so inputs submitted through them had to omit `$system`. Forum posts and old FAQs that mention these commands refer to that site only.

A run killed with signal 15 alone (for example by `timeout` or `kill`) ends its output with `process 0 got signal 15` and no further message; under Slurm a `process 0 got signal 2` line follows, as described above. Give long jobs enough wall time: in a version 25 test, the analytic Hessian of a 22-atom copper complex with L2 basis and a fine grid did not finish within an hour on one core, after a 6-step optimization that had taken most of that hour.

2.7 Input file requirements

The input is a plain text file. Write it in a plain-text editor, not a word processor.

- **Plain ASCII.** Keywords, values and file names should contain ASCII characters only. Do not save the file as Unicode.
- **Unix line endings (LF).** This holds on Windows too. Saving from Word as "plain text" still leaves Windows (CRLF) line endings, and some editors need to be told explicitly to write Unix line endings.
- **Straight quotes.** Write `basis="3z.bas"` with the ASCII double quote. Typographic quotes, as in `basis=≫3z.bas≫`, break the basis-file name.

Users of versions 6–10 reported that an input with CRLF line endings fails in `$molecule`, because the carriage return becomes part of the keyword. The output stops after a line like this, where the keyword `z-matrix` or `cartesian` is shown garbled by the carriage return:

```
 molecule input: ''
' ?matrix
```

Variants seen are `?rtesian` for `cartesian`, `'z-matrix' ?`, and `'cartesian` followed by `' ?` on the next line. A less obvious case: an input edited in Word ran `task=scan` as a plain optimization.

Version 25 behaves the same way: a water input saved with Windows line endings stopped after `molecule input: ''` with the line `'cartesian' ?`, exit status 1. The Windows build `p1.exe` (version 20) stops in the same place, so the input needs Unix line endings on Windows too.

To check and fix the line endings on Linux:

```
file mol.in       # reports "with CRLF line terminators" if the file has Windows endings
dos2unix mol.in
```

Running `dos2unix` on a file that already has Unix line endings does no harm. On Windows, Notepad++ and the FAR Manager editor (Shift+F2, then UNIX (LF)) can convert line endings.

For the full list of input-reading failures and their symptoms, see *The input is not read, or is read wrongly*.

2.8 Basis-set files

The basis sets, their auxiliary (fitting) sets and the ECPs are read from a separate file named by *$control basis*. The set itself is chosen with *set=* in `$molecule`; without it, the default set of the file is used. Without `basis=`, the basis data are read from the input file itself.

Give the basis file as a **full path**:

```
$control task=optimize basis=/home/user/priroda/basis/basis.in $end
```

In version 25, `basis=basis.in` works when the file is in the directory you run from, which in the test was also the directory of the input. One user's parallel run stopped after the node list with `application called MPI_Abort(MPI_COMM_WORLD, 1) - process 0` and worked once the basis file was given with its full path.

- The name can be put in double quotes: `basis="3z.bas"` works. Two single quotes do not work; `basis=''3z.bas''` fails with `error opening file ''3z.bas''`.
- Windows paths work: `basis=D:\NMR\Nature\basis\6-31+G(d)`. In a test, `basis=C:\Priroda\basis.in`, `basis=C:/Priroda/basis.in` and `basis=basis.in` (the file in the directory the run started from) all worked.
- The relativistic file `basis4.in` needs `four=1` in `$control`. Without it the basis input fails: in version 25 the output ends right after `Basis set input: '.../basis4.in'` with the single line `1?`.

Which basis files exist and which sets they contain is described with *$control basis*.

CHAPTER

THREE

GETTING STARTED

This chapter takes you through three things: a single-point energy of water, a geometry optimization of ammonia followed by a Hessian that confirms the structure is a minimum, and the parts of the output you need to read along the way. To download the program, see *Obtaining Priroda*; for the examples you need one Linux executable and `basis.in`. *Installing and running* covers running the program in more detail.

The example inputs were run on Priroda 25 (2025.02.28), and most output excerpts are copied from those runs. That build date matches the file `p4_d` on the download site; the file `p` there is an older build (2023-06-25), so its output can differ from the excerpts here. A few excerpts come from outputs posted by users of older versions; the text says so where it matters.

3.1 Before you start

Three practical points save most first-time failures.

- **Plain text with Unix line endings.** Write inputs as plain ASCII with LF line endings and straight quotes. A file saved with Windows (CRLF) line endings gives errors such as `molecule input: '' 'cartesian ' ?` or `?rtesian`. One user's `task=scan` silently ran an optimization, and CRLF line endings were blamed for it. Convert such files with `dos2unix`.
- **Give the basis file by its full path.** The *basis* keyword names the basis-set file. A relative name works in version 25 when the file is in the directory you run from, also in parallel runs, but relative names have failed in older versions (including parallel runs in version 10), so an absolute path is the safe choice. The examples below use `/home/user/priroda/basis.in`; replace it with the location of `basis.in` on your machine.
- **Atomic numbers are integers.** Each atom line in `$molecule` starts with the atomic number written as an integer (`8`, not `8.0` and not `O`). A real number such as `8.0` gives no error: the program silently reads a different molecule.

A serial run is `p input output`. Parallel runs are described in *Installing and running*.

3.2 A single-point energy

3.2.1 The input

Save the following as `water.in`:

```
$system mem=128 disk=1 $end
$control
 task=energy
 theory=dft
 basis=/home/user/priroda/basis.in
```

(continues on next page)

(continued from previous page)

```
 print=+charges+bonds
$end
$dft functional=PBE $end
$molecule
 charge=0 mult=1
 cartesian
 set=L1
  8    0.000000    0.000000    0.117300
  1    0.000000    0.757200   -0.469200
  1    0.000000   -0.757200   -0.469200
$end
```

Group by group:

- *$system* sets the memory limit in MB and the scratch space. Here `disk=1` means 1 GB of disk for temporary files; a negative value would mean a RAM disk of that many MB. These values are per process.
- *$control* sets the job type and the method. *task=energy* asks for the energy at the given geometry. *theory=dft* is GGA DFT with density fitting, the program's standard method. `print=+charges+bonds` adds Hirshfeld charges and Mulliken populations to the output (see *Charges and bond orders* below).
- *$dft* selects the *functional*. PBE is already the default, so this line only makes the choice visible.
- *$molecule* holds the *charge*, the *multiplicity* (2S+1), the geometry, and the basis set. The geometry is given in Cartesian coordinates (Å), one line per atom: atomic number, then x, y, z. *set=L1* picks the set named `L1` from `basis.in`. `L1` is also the default set of that file, and users compare it to cc-pVDZ in size.

Note

Hybrid functionals such as `PBE1` or `B3LYP` need *theory=ridft*. With `theory=dft`, older versions silently drop the exact exchange, and newer ones abort with no error message. With `theory=tddft`, version 25 still silently drops it. See *$control theory*.

Run it:

```
p water.in water.out
```

3.2.2 What to look for in the output

The excerpts below are from this input run on Priroda 25. The output starts with the version banner, `Priroda 25 (2025.02.28)`, and the memory and disk limits. The line `molecule input:  ''` follows in every run; it is not an error. Then come the coordinates, the `formula:` line and a table of interatomic distances. Check the formula: it is the quickest way to catch a misread `$molecule` group.

Next, a summary box repeats what the program understood from your input. Check that the charge, multiplicity, method and functional are what you meant:

```
 +--------------------------------------
 | Number of atoms                     3
 | Charge of molecule                  0
 | Spin multiplicity                   1
```

(continues on next page)

(continued from previous page)

```
| Number of alpha electrons            5
| Number of beta  electrons            5
| Number of core  electrons            1
| Theoretical Method                 DFT
| Approximation to E(xc)             PBE
| Dimension of wavefunction basis     24
| Dimension of   auxiliary  basis     50
| Hamiltonian: non-relativistic
+--------------------------------------
```

The electron counts are per spin. "Number of core electrons" appears to count core orbitals (here the oxygen 1s). In an RI-MP2 test, removing the core changed this count to 0 and changed the correlation energy.

Two lines then echo the SCF and grid settings, including defaults you did not set:

```
scf: conv = 1.0e-06, 1.0e-03; iter=32,16; xmax = 0.25000; 0s; n1=12
grid: accuracy=1.0e-08,1.0e-05, cut=1.0e-10,1.0e-07, t=-1
```

Next comes the SCF iteration table (columns for the energy, the energy change and the gradient, with indented micro-iterations). Partway through, a `grid:` line marks the switch to the fine integration grid; the gradient jumps there, and in larger molecules the energy can jump too. A successful SCF ends with the line `converged!`.

Warning

If the SCF fails, the program prints `no convergence!` and then **carries on**: it prints an energy, orbitals and properties, and exits normally. The numbers are not usable. (Older versions then also printed `SCF is far from convergence` and stopped.) Always look for `converged!` before trusting a result; *Solving common problems* lists the fixes.

The total energy follows, in hartree, on the line starting with `E =`, together with its components:

```
 E  =     -76.3653913723
 T  =      76.07611549  V  =    -199.28820003  J  =      46.93680394
 Ex =      -8.95038631  Ec =      -0.32925888

 dipole =   0.735449 au (  1.869339 D)
```

If your energy differs from this one in more than the last few digits, check the basis file and the version.

`E` is the number you want. The components `T`, `V`, `J`, `Ex` and `Ec` do not add up to `E`, because the nuclear repulsion energy is not printed on these lines. This is expected and is not an error.

Every run also prints its results as a restart block on lines prefixed with `eng>`. For `task=energy` it holds the energy and the dipole moment; gradient and Hessian runs add more:

```
eng>$Energy
eng> E=-7.6365391372338e+01
eng> D=-2.96281302e-14  6.81315681e-13 -7.35448725e-01
eng>$end
```

3.3 Optimizing a geometry and confirming a minimum

The standard beginner procedure, as taught on the KNC wiki, alternates between optimizations and Hessians:

1. Optimize the geometry.
2. Check that the optimization converged. If it did not, restart from the last geometry.
3. Take the final geometry and compute a Hessian (force constants and vibrational frequencies) there.
4. If any frequency is imaginary, the structure is not a minimum: re-optimize with that Hessian in the input, then compute a new Hessian.
5. When all frequencies are real, read the thermochemistry.

The Hessian check matters most when the start geometry is symmetric. An experienced user reports that Priroda's optimizer rarely leaves imaginary frequencies, except when the starting structure is symmetric and the optimizer keeps that symmetry; then the check is compulsory.

3.3.1 Step 1: optimize

Save the following as `nh3.in`:

```
$system mem=128 disk=1 $end
$control
 task=optimize
 theory=dft
 basis=/home/user/priroda/basis.in
$end
$optimize tol=1e-5 steps=100 $end
$molecule
 charge=0 mult=1
 z-matrix
 set=L1
   7
   1    1    1.02672223
   1    1    1.02672140    2    104.150240
   1    1    1.02671372    2    104.144601   -3    104.150290
$end
```

The third hydrogen is placed by one distance and two angles: the negative index `-3` means "the angle to atom 3" instead of a dihedral.

tol is the convergence threshold on the gradient. The displacement threshold is then set to 100 times that value. The default is `1e-4`; the tighter `1e-5` used in most published examples is a better starting point for a Hessian. *steps* caps the number of cycles (128 by default in version 25; older versions used 50). An optimization that needs far more than 50 steps very likely has something wrong with it, and computing a Hessian first helps in many cases, according to `Dft.rtf`; so don't simply raise the limit. The output echoes all these settings under `Geometry optimization options:`.

Run it with `p nh3.in nh3.out`.

3.3.2 Step 2: check convergence

Each optimization cycle starts with a line such as `Geometry Optimization Step 1:`, followed by the energy, the largest gradient and the largest displacement. When both are below the thresholds, the program prints `OPTIMIZATION CONVERGED in N steps!`. Check for it:

```
grep "OPTIMIZATION CONVERGED" nh3.out
```

Right after the marker, the program prints the optimized geometry as a ready-made `$molecule` group, each line prefixed with `MOL>`, followed by the final energy. For `nh3.in`, version 25 printed:

```
 OPTIMIZATION CONVERGED in 6 steps!
MOL>$molecule
MOL> z-matrix
MOL> set=L1
MOL>   7
MOL>   1    1    1.02753254
MOL>   1    1    1.02744669    2    104.120811
MOL>   1    1    1.02748748    2    104.112983   -3    104.135802
MOL>$end
MOL> Energy =    -56.4988651558
```

The block keeps the coordinate type of the input: a Z-matrix input gives a Z-matrix with the optimized values. The charge and multiplicity seem to be written only when they differ from the defaults (0 and 1): `nh3.in` gives no such line, and an OH^- optimization gave `MOL> charge=-1`. So the block can be pasted into the next input as it is.

3.3.3 Step 3: take the final geometry

Strip the `MOL>` prefix to get a group you can paste into the next input. The following commands copy `nh3.in` to `nh3_freq.in`, switch the task to `hessian`, drop the old `$molecule` group and everything after it, and append the optimized geometry:

```
sed -e 's/task=optimize/task=hessian/' -e '/\$molecule/,$d' nh3.in > nh3_freq.in
sed -n 's/^MOL>//p' nh3.out >> nh3_freq.in
```

The first command assumes that `$molecule` is the last group of the input, as in all examples here. The `Energy = ...` line that ends up after `$end` does no harm: text outside a group is ignored. A normal optimization prints the `MOL>` block only once. A scan prints one `MOL>` block per point (and repeats them as `SCAN>` blocks), so there you pick the block you need; an IRC prints its points as `IRC>` blocks instead.

Note

If the optimization did not converge, there are no `MOL>` lines, and the program prints no message about it: the output just ends after the last step. Do not restart from the original geometry: start from the last one reached. Every step prints the geometry for the next step as a `$molecule` group on lines prefixed with lower-case `mol>`, so this keeps the last such block:

```
sed -n 's/^mol>//p' nh3.out | awk '/\$molecule/{buf=""} {buf=buf $0 "\n"} END{printf "
↪%s", buf}'
```

On an output with no `MOL>` lines, the official `gm` script (below) falls back to the input echo (lines starting with `<$molecule`), which is your *starting* geometry. For a Cartesian input there is no such echo, and `gm` stops with `mol?`. Use the command above instead.

The official helper scripts `ge` and `gm`

The download site ships two small bash scripts for exactly this loop. Both take a job name (with or without the `.in` or `.out` extension) and work on `name.in` and `name.out` in place:

- `gm nh3` replaces the `$molecule` group in `nh3.in` with the optimized one from `nh3.out` and keeps the old input as `nh3.bak`. Like the commands above, it requires `$molecule` to be the last group, because it replaces everything from `$molecule` onwards. It also appends the final `Energy =` line and, when non-zero, the dipole line.
- `ge nh3` appends the `eng>` lines of `nh3.out` (the `$Energy` group: energy, gradient and, after a Hessian run, the Hessian) to the end of `nh3.in`.

So a cycle with these scripts is: run, `gm`, edit the task, run again. Note that `ge` copies every `eng>` line in the output, so use it on a Hessian output, not on an optimization output that has one block per step.

3.3.4 Step 4: compute the Hessian

After step 3, `nh3_freq.in` holds the optimized geometry from `nh3.out` and, after `$end`, its `Energy =` line. Two optional lines are added here by hand: `print=+molden` (for viewing the vibrations, see *Viewing results in Molden*) and a `$thermo` group:

```
$system mem=128 disk=1 $end
$control
 task=hessian
 theory=dft
 basis=/home/user/priroda/basis.in
 print=+molden
$end
$optimize tol=1e-5 steps=100 $end
$thermo sigma=3 $end
$molecule
 z-matrix
 set=L1
   7
   1    1    1.02753254
   1    1    1.02744669    2    104.120811
   1    1    1.02748748    2    104.112983   -3    104.135802
$end
 Energy =    -56.4988651558
```

In a version 25 test, a Hessian run with this input did no optimization: the `$optimize` group did no harm, and it is useful again if you have to go back to step 1. *sigma* is the rotational symmetry number used in the thermochemistry; for ammonia (point group C3v) it is 3. **Priroda does not work it out itself**: without `sigma=` it uses 1, and the entropy comes out too large by R ln σ (2.18 cal/(mol·K) for ammonia). The free energy G is then too low by T·R ln σ, about 0.65 kcal/mol for ammonia at 298.15 K. The default temperature is 298.15 K; other temperatures go in *temperature*.

With `theory=dft` (pure functionals) the Hessian is analytic; the output announces it with `ANALYTIC SECOND DERIVATIVES OF SCF ENERGY`. With `theory=ridft` (hybrids) it is computed numerically.

3.3.5 Step 5: check the frequencies

For a minimum, **all frequencies must be real (positive)**. A single imaginary frequency marks a transition state; several mean a higher-order saddle point. The frequencies are in the table after the line `Harmonic Frequencies`. For ammonia, version 25 printed:

```
*------*---------*--------*---------*---------*--------*-----------*-----------*
| Mode | Freq.   | Mass.  | IR Int. |Raman A. |Raman D.| ROA (90)  | ROA (180) |
*------*---------*--------*---------*---------*--------*-----------*-----------*
|    1 | 1099.30 |   1.18 | 111.146 |   0.000 | 0.0000 |   0.00000 |   0.00000 |
|    2 | 1636.22 |   1.06 |  12.006 |   0.000 | 0.0000 |   0.00000 |   0.00000 |
|    3 | 1636.60 |   1.06 |  12.008 |   0.000 | 0.0000 |   0.00000 |   0.00000 |
|    4 | 3363.03 |   1.03 |   3.508 |   0.000 | 0.0000 |   0.00000 |   0.00000 |
|    5 | 3479.92 |   1.09 |   0.530 |   0.000 | 0.0000 |   0.00000 |   0.00000 |
|    6 | 3480.84 |   1.09 |   0.522 |   0.000 | 0.0000 |   0.00000 |   0.00000 |
*------*---------*--------*---------*---------*--------*-----------*-----------*
```

The columns are the frequency in cm^{-1}, the reduced mass in amu and the IR intensity in km/mol. The Raman columns stay zero in a plain Hessian run; for Raman intensities, run `task=dipole` first and append its `num>` lines (see *Input syntax*). A second table, `Normal Modes (mass-weighted) and Frequencies`, follows with the mode vectors. To print the table:

```
grep -A 14 "Harmonic Frequencies" nh3_freq.out
```

An imaginary frequency is printed with a trailing `i`. Planar ammonia, the transition state of the inversion, gives one:

```
|    1 |  845.22i|   1.21 | 357.748 |   0.000 | 0.0000 |   0.00000 |   0.00000 |
```

`grep "i|" nh3_freq.out` therefore finds the imaginary modes.

3.3.6 Step 6: if a frequency is imaginary, re-optimize

Put the Hessian you just computed into the optimization input. It is printed as a `$Energy` group on `eng>` lines. Starting from a good Hessian, the optimizer can move off the saddle point. In a version 25 test that started exactly on a saddle point (planar ammonia), it reached the pyramidal minimum in 19 steps. Forum users report that one such re-optimization cycle is usually enough.

```
sed -e 's/task=hessian/task=optimize/' nh3_freq.in > nh3_opt2.in
sed -n 's/^eng>//p' nh3_freq.out | awk '/\$Energy/{buf=""} {buf=buf $0 "\n"} END{printf "
↪%s", buf}' >> nh3_opt2.in
```

The second command appends only the last `$Energy` group, after `$molecule`. Do not leave an older `$Energy` group in the file. Then run `nh3_opt2.in` and go back to step 2. (With the official scripts, `ge nh3_freq` appends the group to `nh3_freq.in`; then change its task to `optimize`.)

3.3.7 Step 7: read the thermochemistry

Once all frequencies are real, the end of the Hessian output holds the zero-point energy and a table of thermodynamic functions. The KNC wiki points to the string `ZPVE`. For `nh3_freq.in` (with `sigma=3`), version 25 printed:

```
E =   -56.498867    E0 =   -56.465387

ZPVE = 0.033480 a.u. =   21.0089 kcal/mol

-------------------------------------------------------------------
T =  298.150 K      ln(Q)       S          U          H          G
                              cal/mol*K  kcal/mol   kcal/mol   kcal/mol
-------------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
translational      14.8304    34.4393     0.8887     1.4812    -8.7869
rotational          4.2951    11.5161     0.8887     0.8887    -2.5448
vibrational         0.0057     0.0757    21.0280    21.0280    21.0055
total              19.1312    46.0311    22.8055    23.3980     9.6738
-------------------------------------------------------------------------
```

How to read it:

- `E` is the electronic energy in hartree. `E0 = E + ZPVE` is the energy corrected for the zero-point vibrational energy (here $-56.498867 + 0.033480 = -56.465387$).
- The `U`, `H` and `G` columns are **thermal corrections** in kcal/mol, not total values. They already contain the ZPVE. The program does not print the total enthalpy or free energy; you add them yourself.
- Take the corrections from the `total` row and add them to **E, not E0**. Adding them to `E0` would count the zero-point energy twice:

  H = E + ddH, G = E + ddG, with ddH = `H` (total) and ddG = `G` (total).
- Convert units before adding: 1 hartree = 627.51 kcal/mol. For the table above, $G = -56.498867 \times 627.51 + 9.6738 = -35443.93$ kcal/mol, or $-56.498867 + 9.6738 / 627.51 = -56.483451$ hartree.
- The entropy `S` (cal/(mol·K)) is taken from the `total` row as it stands.

Warning

For a **single atom** the table is wrong. In version 25 (a neon atom), the rotational row has `-inf` for ln(Q) and S and `inf` for G, and it adds RT (0.59 kcal/mol) to U and H, so the total H comes out as 2.07 kcal/mol instead of 5/2 RT = 1.48. Users reported both problems for older versions. Use only the translational row for atoms.

The temperature is printed in the `T =` line. Version 25 uses 298.150 K; outputs posted by users of older versions also show `298.000 K`. Set *temperature* explicitly when you compare results across versions.

3.3.8 The one-run alternative: `task=optimize+hessian`

You can let one job optimize the structure and then compute the Hessian at the converged geometry. In `nh3.in`, change the task line:

```
$control
 task=optimize+hessian
 theory=dft
 basis=/home/user/priroda/basis.in
 print=+molden
$end
```

This is the only combined task in version 25. Two `task=` lines do not chain jobs: users report that only the first one runs. `optimize+hessian` existed in Priroda 2, was reported missing in versions 6 and 8, and is available again from version 9. If the resulting Hessian shows an imaginary frequency, you still need step 6.

3.4 Reading the output

3.4.1 Where things are

The output is plain text, and the results meant for reuse are on lines tagged with a prefix, so `grep` finds them.

Look for	What it is
the summary box (`Number of atoms`, `Theoretical Method`, ...)	what the program understood from the input
`converged!` / `no convergence!`	the end of each SCF
`E =`	total energy in hartree, followed by its components (without nuclear repulsion)
`eng>`	the `$Energy` group: energy `E=`, dipole `D=`, gradient `G=`, and the Hessian after `task=hessian`
`OPTIMIZATION CONVERGED in N steps!`	a converged optimization
`MOL>`	the optimized geometry as a `$molecule` group, then `Energy =`
`Harmonic Frequencies`	the frequency table (`Normal Modes` follows with the mode vectors)
`ZPVE`, `T = ... K`	zero-point energy and the thermochemistry table
`Overlap populations:`	Mulliken populations (`print=+bonds`)
`Atomic partitions of the density:`	Hirshfeld charges (`print=+charges`)
`freq>`	vibrations in Molden format (`print=+molden`, Hessian runs)
`mos>`	orbitals in Molden format (`print=+vectors+molden`)

3.4.2 Charges and bond orders

With `print=+bonds`, SCF runs print Mulliken charges and spin populations under `Overlap populations:`, followed by a table of bond orders. With `print=+charges`, DFT runs print Hirshfeld charges and spins under `Atomic partitions of the density:`. The flags can be combined, as in `water.in`, which gives:

```
Overlap populations:
 atom    charge   spin p.
O1      -0.4167  -0.00000
H1       0.2083  -0.00000
H2       0.2083  -0.00000

  Bond orders:
 O1    |  H1   :  1.07  H2   :  1.07
 H1    |  O1   :  1.07
 H2    |  O1   :  1.07

Atomic partitions of the density:
  atom    charge    spin
  1  O   -0.2905    0.0000
  2  H    0.1451    0.0000
  3  H    0.1451    0.0000
 total   10.0003    0.0000
```

The `total` row of the Hirshfeld block sums the electron populations, not the charges (10.0003, close to the 10 electrons of water).

3.4.3 Viewing results in Molden

The KNC wiki treats Molden as the reference viewer. Open files with the `READ` button in the `Miscellaneous` group.

Vibrations. Run the Hessian with *print=+molden* and cut out the `freq>` lines:

```
sed -n 's/^freq>//p' nh3_freq.out > nh3.freq
```

Open `nh3.freq` in Molden to animate the normal modes.

Orbitals. Add `print=+vectors+molden` to `$control`. The orbitals then appear on `mos>` lines:

```
grep '^mos>' water.out | sed 's/^mos>//' > water.mos
```

The `+molden` part is essential: without it, there are no `mos>` lines. The files are large, and Molden can be slow with them.

Geometry. If the optimized geometry is Cartesian, this one-liner turns the `MOL>` block into an XYZ file that Molden reads (the symbol list covers H to Ar; extend it for heavier elements):

```
sed -n 's/^MOL>//p' job.out | awk 'BEGIN{split("H He Li Be B C N O F Ne Na Mg Al Si P S␣
↪Cl Ar",s," ")}
  NF==4 && $1+0>0 {n++; l[n]=s[$1]" "$2" "$3" "$4}
  END{print n; print "optimized geometry"; for(i=1;i<=n;i++) print l[i]}' > job.xyz
```

For a Z-matrix geometry such as `nh3.in`, the `freq>` file from the Hessian run is the easier way to look at the structure.

pri2mol. The Perl script `pri2mol` by the forum user amge, available from `http://limor1.nioch.nsc.ru/quant/program/pri2mol/`, converts a Priroda output into Molden and XYZ files: geometries, vibrations, orbitals, NMR shifts and density critical points. The download site also has `xm`, a molecule viewer that takes a Priroda input file.

3.5 Where next

- *Input syntax* explains groups, keyword abbreviation, arrays and restart blocks.
- *Installing and running* covers memory, scratch disks and parallel runs.
- The *input reference* describes every known keyword, group by group.
- *Solving common problems* helps when a run fails.

CHAPTER

FOUR

INPUT SYNTAX

`input.txt` describes the input as "a kind of NAMELIST free format". It is close to the GAMESS-US format: keywords are grouped, and each group starts with `$name` and ends with `$end`. This chapter covers the rules that apply to all groups. The individual keywords are in the *input reference*.

A complete input for an NH3 optimization, adapted from the KNC wiki FAQ:

```
$system mem=512 disk=-512 path=/tmp $end
$control
 task=optimize
 theory=dft
 basis=/opt/priroda/basis.in
$end
$optimize tol=1e-5 steps=100 $end
$molecule
 charge=0 mult=1
 z-matrix
 set=L1
   7
   1    1    1.02672223
   1    1    1.02672140    2    104.150240
   1    1    1.02671372    2    104.144601   -3    104.150290
$end
```

4.1 Groups

A group starts with `$` followed by the group name and ends with `$end`. It may sit on one line or span several:

```
$scf conv=1e-7 $end

$scf
 procedure=bfgs
 convergence=1e-7
$end
```

Inside a group, `keyword=value` pairs are separated by spaces or line breaks, and several pairs can share a line (`theory=qm_n3 basis=qm.in`). Blank lines inside a group are allowed.

The order of the groups does not matter, and neither does the order of keywords within a group. The exception is `$molecule`, whose contents are ordered data. (The KNC wiki FAQ also lists `$Energy`, but version 25 reads it with the same keyword reader as the other groups.)

Groups you do not need can be left out; most keywords have sensible defaults. `input.txt` advises changing only what the problem requires.

Two groups can also share a line: `$scf conv=1e-7,1e-3 $end $grid acc=1e-9 $end` sets both values in version 25. Inputs in the sources normally start each group on a new line, which is easier to read.

4.2 Keywords

4.2.1 Case

Group names, keyword names and keyword values are case-insensitive: `$System` and `$system`, `theory=riDFT` and `theory=ridft`, `functional=pbe` and `functional=PBE` are the same. File names are the exception, because Linux file names are case-sensitive. This applies to `basis=`, `save=`, `path=` and other file names, such as `file=` in `$dynamic` and `$uniq`.

4.2.2 Abbreviations

Within a group, a keyword can be shortened to a prefix. The example in `input.txt` is `mem` for `memory`. Inputs in the sources use, among others:

Short form	Keyword
`mem=`	*$system memory*
`conv=`, `iter=`, `proc=`	*$scf convergence*, *iterations*, *procedure*
`func=`	*$dft functional*
`acc=`, `accur=`	*$grid accuracy*
`tol=`, `cart=`, `val=`	*$optimize tolerance*, *cartesian*, *value*
`disp=`	*$d2edr2 displace*
`nuc=`	*$control nucleus*
`rout=`	*$integral routine*

Keywords belong to their group. The same name can mean different things in different groups: `steps` in *$optimize* is the maximum number of optimization cycles, while `steps` in *$d2edr2* selects displacement steps of a numerical Hessian.

Warning

A prefix must stay unambiguous within its group. In `$scf`, `co` matches both `convergence` and `core`; in version 25, `$scf co=1e-7 $end` stopped with `unknown name e-7`, a message that does not point to the cause. In inputs you keep or share, prefer the full keyword names; they are also easier to search for.

Do not give the same keyword twice in one group, even in different spellings. In version 6, an `$optimize` group with both `step=` and `steps=` took the maximum number of steps from `step`.

A misspelled or unknown keyword name stops the run right after the basis-set echo, with a message naming the group and the keyword:

```
*** error in '$scf' : unknown name nosuchkeyword
```

Only groups that the task reads are checked. In version 25, an unknown keyword in `$optimize` stopped an optimization (`unknown name coordinated`) but was ignored in an energy run, which does not read `$optimize`. A clean energy run therefore does not prove that the rest of the input is correct.

The same message can appear for a keyword you spelled correctly. In version 25, `convergence=` with a single value, followed by `iterations=`, stops the run with the name of `iterations` cut short:

- `$scf convergence=1e-7 iterations=250 $end` printed `unknown name terations` on the machine of the first test rounds, also with the two keywords on separate lines;
- the same input printed `unknown name erations` in later runs of the `p4_d` executable on another machine. There the message was the same for every value tried (`1e-5`, `1e-6`, `1e-7`, `1e-10`, `0.000001`, `1.0e-6`, and `1e-6,` with a trailing comma), and `iter=50` gave `unknown name er`.

Other keywords after a one-value `convergence=` were read normally (`procedure`, `restrict`, `xmax`, `diagm`, `temperature`, `fraction`, `core`, `eemin`, `aw`), but `n1=` stopped the run with `no '$end' found, 'convergence'`. The problem disappears when `convergence` gets both of its values (`convergence=1e-6, 1e-3 iterations=50`) or comes after `iterations`. Give `convergence` both values, or put it last in the group; see *$scf convergence*.

4.3 Values

Scalars
Integers (`steps=100`), reals in plain or exponent notation (`trust=0.2`, `tol=1e-5`) and words (`task=optimize`). A sign is allowed: `charge=+1`.

Lists
Values separated by commas, for keywords that take several numbers: `tol=1e-5,1e-4`, `mem=2048, 256`, `temperature=203, 223, 248, 298`. Spaces after the commas are allowed, and some lists are also written with spaces between groups of values (`fix=1,1,2,0,0 3,2,1,4,3`). If you give fewer values than the keyword holds, the rest keep their defaults: `$scf conv=1e-10` sets only the SCF threshold and leaves the CP-SCF threshold at `1e-3`.

Indexed arrays
`name(i)=a,b,...` assigns values to an array starting at element i and leaves the other elements unchanged. `$guess na(85)=86,85 $end` sets element 85 of `na` to 86 and element 86 to 85, which swaps two orbitals. The same form is used for *$optimize value* (`val(5)=2.0,2.1`), *$nmr standard* (`standard(6)=181.9921`) and *$atoms core*.

Flag strings
$control print takes a string of options, each switched on with `+` or off with `-`: `print=+vectors+molden`, `print=-internals+molden`. The leading `+` may be omitted (`print=charges+bonds`).

Quoted strings
A file name or path may be put in double quotes. The quotes are optional: `basis="3z.bas"` and `basis=basis.in` both work, as does `path="/scratch/priroda"`. Use the straight ASCII double quote `"`. Typographic quotes break the name, and two single quotes do not count as a double quote: `basis=''3z.bas''` fails with `error opening file ''3z.bas''`. Avoid spaces and non-ASCII characters in file names, directory names and the host name.

4.4 The `$molecule` group

$molecule is read as ordered data rather than as keyword pairs:

1. `charge=` and `mult=` (spin multiplicity 2S+1), both optional, with defaults 0 and 1;
2. one of the words `cartesian` or `z-matrix`;
3. one line per atom, optionally preceded by `set=` lines that choose the basis set for the atoms that follow.

The first items may share the opening line:

```
$molecule charge=0 mult=1 cartesian
 set=L1
  8    0.000000    0.000000    0.117300
  1    0.000000    0.757200   -0.469200
  1    0.000000   -0.757200   -0.469200
$end
```

Each atom line begins with the atomic number written as an integer. Cartesian coordinates are in Å unless `unit=b` is given. The Z-matrix forms, mixed Cartesian and Z-matrix input, isotope masses and per-atom basis sets are described in the *$molecule* reference.

4.4.1 Several `$molecule` blocks

Some tasks read more than one `$molecule` group from one input:

- `task=transit`, `task=saddle` and `task=path` take two blocks, the start and the end structure. The atoms must be numbered the same way in both. `task=saddle` is not the same as *$optimize saddle=1*, the transition-state search from a stored Hessian. In a version 25 test, `task=saddle` with the two mirror-image structures of ammonia ran a two-stage search (a one-dimensional maximization between the structures, then a two-dimensional saddle search) and stopped at the planar midpoint, 0.00003 hartree above the fully optimized transition state. `task=transit` with the same input crashed with a bus error after printing its internal coordinates. `task=path` was not tested.
- `task=uniq` takes any number of blocks and filters out duplicate structures; the official example has 157.

These tasks are undocumented; see *$control task* and *$uniq*.

4.5 Text outside groups

Everything outside a `$name ... $end` group is ignored. There is no documented comment character, but this rule serves the same purpose, and the official example inputs use it to switch input off:

- **Delete the `$`** to disable a whole group. `integral dir=1 $end` in the official 2023–2025 example inputs is plain text, not an `$integral` group.
- **Move keywords after `$end`** to disable them. In the example below, `v=` and `h=` are outside `$dynamic` and have no effect.
- **Keep notes after `$end`.** Helper scripts append result lines such as `Energy = ...` after the `$molecule` group, where they do no harm.

```
integral dir=1 $end
$dynamic
 e=0.001
 d=0.03
 n=1000,100
$end
 v=500.0
 h=0.1
$molecule charge=0 mult=1 cartesian
 set=L1
  8    0.000000    0.000000    0.117300
  1    0.000000    0.757200   -0.469200
  1    0.000000   -0.757200   -0.469200
```

(continues on next page)

(continued from previous page)

```
$end
 Energy =   -76.36539137
```

In version 25 a line of text before the first group and a disabled group with its leftover `$end` (`integral dir=1 $end`) are both ignored: the result is identical to the same input without them. Text inside a group is parsed as keywords, so do not put notes there.

A group opened with `&` instead of `$` is ignored like any other text: in version 25, `&dft func=BLYP $end` left the functional at PBE, with no message. Always use `$`.

4.6 Appending results of earlier runs

Priroda writes reusable results into the main output file. Each such line carries a prefix, for example `eng>` or `num>`. Strip the prefix and append the lines to the next input. The program has no separate restart file for these data; SCF orbitals are the exception, saved with *$control save*.

4.6.1 `$Energy` from `eng>` lines

A run prints its energy and dipole, plus the gradient when one is computed and, after `task=hessian`, the Hessian, as an *$Energy* group on lines prefixed with `eng>`:

```
eng>$Energy
eng> E=-2.894457077327e+04
eng> D= 0.00000000e+00  0.00000000e+00  0.00000000e+00
eng> G= 3.53877831e-09  1.65504544e-08 -3.64771621e-07
eng>   -1.31128901e-07 -6.73350427e-09 -2.19780956e-06
eng>    1.27590122e-07 -9.81695012e-09  2.56258122e-06
eng>$end
```

To reuse it, remove the prefix and append the block to the end of the next input, after `$molecule`:

```
sed -n 's/^eng>//p' mol.out >> mol.in
```

The `ge` script on the official site does the same. When the output contains a `NUMERICAL AND ANALYTICAL GRADIENT` section, it ignores everything before that section.

Warning

An optimization prints an `eng>` block at every step, and the command above appends all of them. Append only the block you need, normally the one from a `task=hessian` run. If the input holds several `$Energy` groups, the program uses the **last** one, even when an earlier one matches the geometry.

A stored Hessian is required by *$optimize saddle=1* and `task=irc`. It also helps difficult minimizations, and it lets you redo the vibrational analysis or the thermochemistry without recomputing.

4.6.2 `num>` lines

Partial results of a numerical calculation are printed on lines prefixed with `num>`. There are two uses:

- A numerical Hessian split over several runs with *$d2edr2 steps*. Collect the `num>` lines of all partial outputs into one input, add `$d2edr2 steps=-1 $end`, and run once more:

```
grep -h "^num>" mol.out1 mol.out2 mol.out3 mol.out4 | sed 's/^num>//' >> mol.in
```

The lines hold numbered `$Energy000`, `$Energy001`, ... groups, one per displaced geometry. The combining run does no SCF of its own. The command given in `input.txt`, `sed "s/\.*num>//g"`, leaves the file names that `grep` adds for several files at the start of each line; use `grep -h` as above.

- Raman intensities. A `task=dipole` run writes `num>` lines; append them to the input of a `task=hessian` run, and the `Raman A.` and `Raman D.` columns of the frequency table are filled in.

4.6.3 Optimized geometry from `MOL>` lines

After `OPTIMIZATION CONVERGED`, the final geometry is printed as a ready-made `$molecule` group on lines prefixed with `MOL>`. Strip the prefix to continue from that geometry. The `gm` script on the official site replaces the `$molecule` group of the input with it. Because `gm` replaces everything from `$molecule` to the end of the file, keep `$molecule` as the last group of your inputs. Anything appended after it, including an `$Energy` group with a Hessian, is deleted by `gm`; append such blocks again afterwards.

An optimization that runs out of steps prints no `MOL>` lines in version 25. The geometry of each step is printed on lines prefixed with lowercase `mol>`; the block from the last step is the geometry to restart from. `gm` finds neither `MOL>` lines nor an input echo in such an output and stops with `mol?`, so strip the last `mol>` block by hand.

4.7 Common mistakes

Non-integer atomic numbers
`25.0` or `8.0` instead of `25` or `8` in `$molecule`. The atom lines are read as a stream of numbers, so the rest of the group shifts. In version 25 this gives no error at all: water written with `8.0` and `1.0` ran as an oxygen atom plus three empty centres with Z=0, and printed a meaningless energy. Older versions were reported to stop with `illegal charge/multiplicity`. Check the `formula:` line near the top of the output. Element symbols are not accepted either: in version 25, water written with `O` and `H` stopped after `molecule input:  ''` with the line `'o' ?`.

Typographic quotes
Word processors and some web pages turn `"` into curly or angle quotes, which break file names in `basis=` and `path=`. Retype them as straight quotes.

Two `task=` lines
A second `task=` line does not start a second job. In version 25 the last one wins: with `task=optimize` and then `task=hessian` in `$control`, only the Hessian was computed, and nothing was printed about the first line. In version 6 users reported that only the first task ran. `task=optimize+hessian` is the only combination the program understands. Run other sequences as separate jobs.

Windows line endings (CRLF)
The run stops in `$molecule` with `molecule input:  ''` followed by a garbled `' ?matrix` or `?rtesian`. Convert the file with `dos2unix`; see *Installing and running*.

Misspelled task or theory name
Gives no error. An unknown `task=` value sets up the run and stops quietly after the SCF and grid settings; an unknown `theory=` value runs Hartree–Fock, with the misspelled name in the summary box. Check the `Theoretical Method` line and that the output goes on to an SCF.

Hybrid functional without `theory=ridft`
Older versions ran but gave wrong results. With `theory=dft`, version 25 stops after the basis-set echo with no message in the output file (stderr shows only `MPI_Abort`); with `theory=tddft` it still runs without the exact exchange and gives wrong energies. See *$control theory*.

`basis4.in` **without** `four=1`
: The relativistic basis file needs `four=1` in `$control`. Without it, version 25 stops right after the line `Basis set input:` with the single line `1?`.

More input problems, with their exact error messages, are listed in *The input is not read, or is read wrongly*.

CHAPTER

FIVE

INPUT REFERENCE

Group	Purpose
$system	Memory, disk and scratch-directory limits.
$control	Job type, level of theory, basis-set file, Hamiltonian and printing.
$molecule	Charge, multiplicity, geometry (Cartesian and/or Z-matrix) and per-atom basis sets.
$guess	Initial guess for the SCF vectors.
$scf	Spin restriction, convergence, iteration limits and SCF solver.
$dft	Exchange–correlation functional.
$grid	Adaptive integration grid for DFT.
$integral	Integral evaluation for RI methods.
$atoms	Per-element core definitions for correlated (HF-based) methods.
$optimize	Geometry optimization, saddle-point search, IRC and relaxed scans.
$d2edr2	Numerical Hessian by finite differences of the gradient.
$thermo	Thermochemistry after a Hessian calculation.
$nmr	Reference shieldings and extra points for NMR shielding calculations.
$Energy	Results of an earlier run (energy, gradient, Hessian, dipole data), read back as input.
$dynamic	Molecular dynamics with a thermostat.
$uniq	Duplicate-structure filtering over several `$molecule` blocks.

5.1 $system

Memory, disk and scratch-directory limits.

All limits apply **per MPI process**. For example, `disk=10` on four processes can use 40 GB. For GGA DFT, the v5 documentation and a v8 benchmark report that most of the memory and all of the disk storage are used by the master process (rank 0). RI-MP2 and the other correlated methods write large scratch files on every process. In version 25, a two-process DFT run left one scratch file per process.

5.1.1 memory

Type: list
Default: `256`
Units: MB
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §4.1 (two values); 06 §3 (estimate); 03 §4, 04 #196 (per process); 08 t02, t58a, w04 (same echo in the Windows version 20)

Default: echoed by version 25 when `$system` is omitted.

Memory limit in megabytes. A second value sets the limit for the other MPI ranks: `mem=2048,256` gives rank 0 2048 MB and every other rank 256 MB. A single value applies to all ranks. The output echoes the

result as `Memory:  <mem>m + <|disk|>m (<mem2>m + <|disk2|>m)`.

Note

For DFT, the v5 documentation estimates the memory as about $8 \cdot N^2 \cdot 3$ bytes for energy and gradient, and twice that for an analytic Hessian, where N is the basis dimension. `Dft.rtf` gives an old default of 16 MB. Without a `$system` group, version 25 echoes `Memory:  256m + 512m (256m + 512m)`, the same form that `mem=512 disk=-512` gives. This is read as 256 MB of memory plus a 512 MB RAM disk per process *(inferred from the echo format)*.

5.1.2 disk

Type: list
Default: `-512`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf (one file per GB); 01 §4.1; 04 §0, #468; 03 §3, §17; 08 t02, t58a, t62, t64; 06 §3, 08 Slurm s02 (scratch per process)

Default: a 512 MB RAM disk *(inferred from the version 25 echo `Memory:  256m + 512m` when `$system` is omitted)*.

Positive values set the disk space in gigabytes; one scratch file is opened per gigabyte, which sidesteps per-file size limits. Negative values set the amount of memory, in megabytes, used to emulate disk (a RAM disk). A second value applies to the other MPI ranks, as for `memory`; the official example inputs typically give them `-1`.

Note

With a RAM disk, `memory` plus —`disk`— (times the number of processes) must fit in physical RAM, or the run fails with `malloc(...) failed`. Example for a 24 GB node running a single process: `memory=9000 disk=-14000`. On one cluster, an RI-MP2 job on 8 cores with scratch in RAM ran 6× faster than on 2 cores with scratch on disk (a v8 benchmark; core count and scratch location both changed). In version 25, `disk=-512` was echoed as `Memory:  512m + 512m` with no `Disk:` line, `disk=1` as `Disk:  1g '/tmp'`, and conventional MP2 with `disk=-1` stopped with `0:  out of disk space (grid_W)`.

Example:

```
$system mem=2048,256 disk=-256,-1 $end
```

5.1.3 path

Type: string
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 03 §2; 04 #465; 08 t01, t64, w02a-d, w03

Default: the `TMPDIR` environment variable, otherwise `/tmp` (version 25). `input.txt` says the current working directory.

Directory for temporary disk files. For parallel runs you can give a colon-separated list with one path per process: `path=/tmpa:/tmpb:/tmpc:/tmpd` makes process 1 write to `/tmpa`, process 2 to `/tmpb`, and so on. Those directories should be on different physical drives. The value may be double-quoted.

Note

For all methods except GGA DFT, performance degrades “terribly” if more than one process reads and writes large files on the same physical drive. Never put temporary files on NFS. Version 25, run without `path=`, used `TMPDIR` when it was set and `/tmp` when it was not (echo `Disk: 1g '/tmp'`). The Windows `p1.exe` (version 20) does the same, and as Windows has no `/tmp` the run then stops at `0: opening file /tmp/...`. Use `path=.`, set `TMPDIR` to a Windows path without quotes (`set TMPDIR=%TEMP%` in `cmd.exe`), or use a RAM disk.

Example:

```
$system path=/tmpa:/tmpb:/tmpc:/tmpd $end
```

5.2 $control

Job type, level of theory, basis-set file, Hamiltonian and printing.

5.2.1 task

Type: enum
Default: `gradient`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf (default gradient, check); 02 §3.2 (pri: default gradient); 03 §5; 04 #69-#73, #145, #478, #497; 01 §4.2; 08 t02, t18, t26, t34, t36, t37; 09; 08 t50, t57a; 08 t13, t14, t56, t69a-f

Job type.

Value	Meaning
`energy`	Energy at the given geometry.
`gradient`	Energy and nuclear gradient. Costs little more than the energy. (**default**)
`hessian`	Second derivatives (force constants) followed by vibrational analysis and thermochemistry.
`optimize`	Geometry optimization, including saddle points with `$optimize saddle=1`.
`optimize+hessian`	Optimization followed by a Hessian at the final geometry. The only combined task that works. Present in v2 and v9 and later, absent in v6.
`irc`	Intrinsic reaction coordinate. Needs a Hessian at the TS in the input (`$Energy`). Each point is printed on `IRC>` lines with its distance and energy.
`scan`	Relaxed potential-energy-surface scan over the coordinates in `$optimize fix`. The structures and energies are summarized on `SCAN>` lines at the end.
`nmr`	NMR shielding tensors. Works only with `theory=dft`: with `ridft`, version 25 silently runs only the ground-state SCF.
`dipole`	Numerical dipole polarizabilities and their derivatives with respect to nuclear coordinates (used for Raman intensities), from SCF runs in finite electric fields. The result is a `num>` block.
`check`	Not a task name in the version 25 program. Like any unrecognized task, `task=check` sets up the run (geometry, basis sets, SCF and grid settings) and stops before the SCF without a message, which makes it a usable input check. *(undocumented)*
`extremal`	Gradient-extremal path (v9 banner "Calculation of Gradient Extremal Path"); slow. *(undocumented)*
`transit`	Synchronous transit in internal coordinates. Needs two `$molecule` blocks (start and end). Reported working in v8. In version 25 a test with two ammonia structures crashed with a bus error after printing the internal coordinates (08 t81d). *(undocumented)*
`saddle`	Approximate saddle-point search from two `$molecule` blocks (start and end). Version 25 prints `Approximate Saddle Point Search`, then a one-dimensional maximization between the two structures and a two-dimensional saddle search; for the two mirror images of ammonia it stopped at the planar midpoint (08 t81c). *(undocumented)*
`path`	Minimum-energy-path optimization from two `$molecule` blocks (start and end). *(undocumented)*
`conical`	Conical intersection of states 0 and 1 (v9 banner). Untested. *(undocumented)*
`anharmonic`	Anharmonic corrections to vibrational energies; slow. *(undocumented)*
`opt_n1`	Optimizer for very large systems (used for 8060 atoms with `theory=qm_n2`). Pairs with `$optimize mod=1`. *(undocumented)*
`thermostat`	Molecular dynamics with a thermostat; see `$dynamic`. In version 25 it ran with `dft`, `ridft`, `rimp2`, `ccsd`, `tddft` and `qm_n3`. *(undocumented)*
`uniq`	Apparently filters duplicate structures among several `$molecule` blocks; see `$uniq` (inferred from the official example; untested). *(undocumented)*

Note

Two `task=` lines do not chain jobs: only the first one runs. Apart from `optimize+hessian`, sequences such as optimize→NMR need separate runs or an external driver script. **A misspelled task name**

gives no error: version 25 sets up the run and stops quietly, as for `check`. `critical` is not a task name in version 25, so it stops silently like any unknown task. Users found that `task=crytical` gave nothing; density critical points come from `print=+density`. `task=QM` appears only in the header of the shipped `qm.in` parameter file and is not a user task. The version 25 program contains further task names (`irc4`, `pathx`, `pathr`, `chain`, `walk`, `steric`, `polarizability`, and others) that are untested or need data that is not distributed.

See also: *saddle ($optimize)*, *points ($optimize)*, *print ($control)*

5.2.2 `theory`

Type: enum
Default: `dft`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf (default dft); 06 §3 (v5 ab initio list); 03 §5 (Evgeniy 2011 test); 04 #96, #99, #307-#310; 01 §3.1, §4.2; 08 t02, t08, t17, t31-t33, t38, t39, t46; 08 t50b; 08 t78a-d; 09

Theoretical method.

Value	Meaning
`dft`	GGA DFT with density fitting. Hybrid functionals are **not** applied here: see notes. (**default**)
`ridft`	Hybrid DFT with exact exchange via the resolution-of-identity (RI) approximation. Required for hybrid functionals; the default functional is then `PBE1`. Hessians are numerical.
`DFT_n4`	Newer DFT variant, used in the official example inputs with `func=LPBEVV` (2025). Runs in version 25 (summary box: `DFT_N4`); what it changes is not documented. *(undocumented)*
`tddft`	GGA time-dependent DFT (no hybrid TDDFT).
`rimp2`	RI-MP2. Hessians are numerical.
`hf`	Hartree–Fock.
`rihf`	RI Hartree–Fock.
`cis`	CIS (RHF/UHF). Prints a table of excited states; `E =` is the total energy of the first excited state. Slow and disk-hungry.
`mp2`	Conventional MP2 (RHF, UHF, ROHF). Very disk-hungry.
`mp3`	MP3.
`mp4dq`	MP4(DQ).
`mp4sdq`	MP4(SDQ).
`mp4sdtq`	MP4(SDTQ) (RHF).
`ccs`	CCS. Returns the HF energy, as expected: CCS adds no correlation energy to an HF reference.
`ccd`	CCD.
`ccsd`	CCSD (RHF/UHF).
`ccsd4t`	CCSD(T).
`ccsdsd`	CCSD for an excited state, closed-shell (RHF) only; the state is chosen with `state=`, as for `cis` and `tddft`. Version 25 first solves CCSD for the ground state, then the excited state, and reports the excited state's total energy in `eng`>. Without `state=` it gives the CCSD ground-state energy. For an open-shell molecule it silently stops after UHF (no CCSD, exit 0). Described as linear-response CCSD for excited states (RHF) by the v5 keyword page.
`qm_n3`	Laikov's semiempirical QM model. Needs `basis=qm.in` and no `set=`; parameters exist for H, Li–F, Na–Cl, Sc–Cu. Runs in version 25, with a short output of its own (no summary box, no `E =` line). *(undocumented)*
`qm_n2`	Earlier variant of the QM model, used for very large systems with `task=opt_n1`. *(undocumented)*

Note

A misspelled theory name gives no error: version 25 prints it in the summary box and runs Hartree–Fock.

Hybrid trap. Hybrid functionals (`PBE1`, `B3LYP`, ...) take effect only with `theory=ridft`. With `theory=dft` or `tddft`, older versions silently drop the exact exchange. In version 25, `theory=dft` stops right after printing the basis sets, with no error message in the output and exit status 1. `theory=tddft` still runs: it reports `PBE1` in the summary box but leaves out the exact exchange, so the energies are wrong (water: `Ex` -6.61 instead of about -8.95 hartree, ground-state total energy about 2.2 hartree above PBE), and the run ends normally.

The ab initio list comes from the v5 keyword page and a 2011 user test (v6), which was energy-only. In version 25, water energies run to completion with `hf`, `rihf`, `cis`, `mp2`, `mp3`, `mp4dq`, `mp4sdq`, `mp4sdtq`,

`ccs`, `ccd`, `ccsd` and `ccsd4t`; an `hf` optimization also worked, so HF gradients exist. For `ridft` and `rimp2`, `task=hessian` works by finite differences of the gradient (`NUMERICAL COMPUTATION OF SECOND DERIVATIVES`); for `dft` it is analytic. These were reported **not** working in v6: `mp4`, `rimp4`, `rimp4sdtq`, `rimp4sdq`, `riccsd`, `riccsd4t`. They either crash or return the HF energy. The version 25 program contains many more method names (`MCSCF`, `CASSCF`, `CISD`, `LCCD+MP2`, ...); none has been tested.

See also: *functional ($dft)*

5.2.3 `state`

Type: list
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t33

For excited-state methods. The first value is the excited state n whose properties are computed; the second is the number of states N included in the iterative solution of the equations (`state=n,N`). In version 25, `theory=tddft` with `state=1,5` printed all five states in an `Electronic Spectrum:` table (excitation energy, wavelength, transition moment, oscillator strength), and `E =` was the total energy of state 1.

5.2.4 `basis`

Type: string
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf; 01 §4.11, §6; 03 §5, §6; 04 §0, #257; 08 t03

Default: basis-set data are read from the input file itself.

Name of the file containing the basis-set data (and ECPs, or QM parameters for `qm.in`). The name may be double-quoted, which is needed when the path contains special characters. Use straight quotes: typographic quotes break parsing.

Note

Shipped files include `basis.in` (Λ/L family, default set `L1`), `basis4.in` (scalar-relativistic L family; needs `four=1`), `3z.bas` ($Z \leq 54$; recommended for DFT only, as users report large errors with ab initio methods), `sbk.bas` (SBK ECPs, up to $Z = 86$), `b2*.in`, `b2u*.in`, `b4s*.in`, and `qm.in`. In version 25 a relative name such as `basis=basis.in` works when the file is in the directory the program is started from. An absolute path is still the safest choice for parallel and batch runs.

See also: *set ($molecule)*, *four ($control)*

5.2.5 `four`

Type: switch
Default: `0`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §3.3; 04 #207-#208, #336; 08 t35, t49

Four-component option.

Value	Meaning
`0`	Non-relativistic theory. (**default**)
`1`	Scalar-relativistic theory (K. G. Dyall, J. Chem. Phys. 100 (1994) 2118–2127). The output reports "Hamiltonian: scalar-relativistic".

Note

Required with the relativistic basis file `basis4.in`. Without it, version 25 stops right after the `Basis set input:` line with `1?`.

See also: *nucleus ($control)*, *d1small ($scf)*

5.2.6 `nucleus`

Type: enum
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §4.2; 06 §1; 08 t35; 09 (dual)

Default: `finite` for four-component theory, `point` for non-relativistic theory.

Nuclear model. The official example inputs write it as `nuc=finite`.

Value	Meaning
`finite`	Gaussian nuclear model (L. Visscher, K. G. Dyall, At. Data Nucl. Data Tables 67 (1997) 207–224).
`point`	Point-nucleus model.
`dual`	Found in the version 25 program; its meaning is unknown. *(undocumented)*

Note

A basis optimized for a point nucleus must not be used with the finite-nucleus model, and vice versa. With `four=1` and no `nucleus=`, version 25 prints `Finite nucleus model (1):` with the Gaussian parameters of each element.

See also: *four ($control)*

5.2.7 `print`

Type: flags
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf; 06 §3; 04 #376, #460, #498-#502, #509; 08 t01, t45

Printing options. Flags are appended one after another; `+` switches an option on and `-` switches it off. The leading `+` can be omitted (`print=charges+bonds`).

Value	Meaning
`+charges`	Hirshfeld charges (DFT methods), printed as "Atomic partitions of the density". With `theory=hf`, version 25 stops after the SCF with `0: read 'Basis2'` and exit 1.
`+bonds`	Mulliken populations and bond orders (SCF only), printed as "Overlap populations".
`+esr`	ESR parameters.
`+vectors`	SCF vectors.
`+molden`	MOLDEN data: vibrations on `freq>` lines for `task=hessian`; with `+vectors+molden` also orbitals on `mos>` lines (`[Molden Format]`, `[Atoms] AU`, `[5D]`, `[GTO]`, `[MO]` sections).
`+density`	Critical points of the electron density (QTAIM). Combine with `+charges` to label them. *(undocumented)*
`-internals`	Suppress the printout of internal coordinates. *(undocumented)*

Example:

```
$control print=+vectors+molden $end
```

5.2.8 derivative

Type: enum
Default: *none stated*
Status: undocumented · verified on 25 (2025.02.28)
Sources: 09; 08 t53

Default: analytic second derivatives where the method has them (GGA DFT), numerical otherwise.

How `task=hessian` computes the second derivatives. Not in any written source; found in the program and tested in version 25, where `derivative=numeric` with `theory=dft` gave the same water frequencies as the analytic Hessian to within 0.2 cm^{-1}.

Value	Meaning
`analytic`	Analytic second derivatives.
`numeric`	Second derivatives by finite differences of the gradient (`NUMERICAL COMPUTATION OF SECOND DERIVATIVES`).

See also: *steps ($d2edr2)*

5.2.9 save

Type: string
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf; 04 #306; 08 t27

File name for saving SCF vectors. With `$guess read=1` the vectors are read back from the same file. This is the basis of the SCF restart technique. In version 25, a run with `$guess read=1` prints `Initial Guess (-1)` and `vectors from '<file>'`; the normal guess is printed as `Initial Guess (2)`.

See also: *read ($guess)*

5.3 $molecule

Charge, multiplicity, geometry (Cartesian and/or Z-matrix) and per-atom basis sets.

Unlike the other groups, `$molecule` holds **ordered data**. It starts with the optional `charge=` and `mult=` (defaults 0 and 1), then one of the keywords `cartesian` or `z-matrix`, followed by one line per atom.

Cartesian lines: `Z x y z`. **Z is the atomic number written as an integer**: `8`, not `8.0` and not `O`. The atom lines are read as a stream of numbers, so a value such as `8.0` shifts everything after it. In version 25 this gives **no error**: the run continues with a wrong molecule (extra atoms with Z=0) and a meaningless energy. Check the `formula:` and atom-count lines at the top of the output. The coordinates are in Å unless `unit=b` is given.

Z-matrix lines give the atomic number `Z_i` of atom i, then up to three reference atoms j, k, l, each followed by an internal coordinate. They can combine distances, angles and torsions in four ways:

Form	Meaning
`Z_i j r_ij k a_ijk l b_ijkl`	distance, angle, torsion
`Z_i j r_ij k a_ijk -l a_ijl`	distance and two angles
`Z_i j r_ij -k r_ik l b_ijkl`	two distances and torsion
`Z_i j r_ij -k r_ik -l r_il`	three distances

Distances are in Å (bohr with `unit=b`), and angles and torsions are in degrees. In a pure Z-matrix the first atom gives only its atomic number, the second only a distance, and the third two internal coordinates. The two formats can be mixed: put the Cartesian atoms first, then the remaining atoms as a Z-matrix.

The program does not use or impose point-group symmetry unless the newer `sym=` option is given. Users report that ghost atoms (basis functions without a nuclear charge), which counterpoise (BSSE) corrections need, are not available. In version 25, atoms with Z=0 are accepted but carry no basis functions, so they are not ghost atoms. NICS probe points go in `$nmr points=` instead. Some tasks (`transit`, `saddle`, `path`, `uniq`) read **several** `$molecule` blocks from one input. Text after `$end`, such as the `Energy = ...` lines that helper scripts append, is ignored. The line `molecule input: ''` near the top of every output is normal and not an error.

5.3.1 charge

Type: integer
Default: `0`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf (default); 08 t29

Molecular charge.

5.3.2 mult

Type: integer
Default: `1`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf (default); 08 t10

Spin multiplicity, 2S+1.

5.3.3 mass

Type: real
Default: *none stated*
Units: amu
Status: current · not verified
Sources: input.txt

Optional atomic mass for isotopes, written as `mass=` after an atom's coordinates on the same atom line, in both Cartesian and Z-matrix input. Not tested in version 25.

Example:

```
1  0.0  0.0  0.96  mass=2.014
```

5.3.4 set

Type: string
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 03 §5 (default set); 01 §4.11, §6.2; 06 §1; 08 t01 (set=L1), t70a (set=L2)

Default: the set named by `default=` in the basis-file header (`L1` in `basis.in`).

Basis set for the atoms that follow. The name must match one in the basis-set file. It can appear several times in the group. The orbital basis is paired with its own auxiliary (fitting) basis, so you cannot combine arbitrary orbital and auxiliary sets.

Example:

```
$molecule
 z-matrix
 set=L2
   8
 set=L1
   1    1    0.96252725
   1    1    0.96252725    2    102.943872
$end
```

See also: *basis ($control)*

5.3.5 unit

Type: string
Default: *none stated*
Status: undocumented · verified on 25 (2025.02.28)
Sources: 01 §4.7; 08 t47

Length unit of the coordinates. The default unit is Å. In version 25, water given in bohr with `unit=b` gave the same geometry and energy as in Å.

Value	Meaning
`b`	Coordinates in bohr.

5.3.6 sym

Type: list
Default: *none stated*
Status: undocumented · verified on 25 (2025.02.28)
Sources: 01 §4.7; 04 #481; 08 t15; 08 t30a

Used as `sym=1,0.01` in the official 2023–2025 example inputs, apparently to enable symmetry with tolerance 0.01. Newer versions report the detected point group (`symmetry: D6h`). In version 25, `sym=1,0.01` on a slightly distorted NH3 printed `symmetry: C3v 6, unique: 2` and echoed the geometry symmetrized. On its own it does not keep the symmetry during an optimization; add `$optimize sym=1` for that.

See also: *symmetry ($optimize)*

5.4 $guess

Initial guess for the SCF vectors.

5.4.1 read

Type: switch
Default: `0`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf (default); 04 #306; 05 §5, §7; 08 t27, t71b

Default: `read=0` is the default in `Dft.rtf`. Version 25 without `read=1` prints `Initial Guess (2)`, the normal guess.

Source of the starting SCF vectors.

Value	Meaning
`1`	Read SCF vectors from the file named by `$control save=`.
`0`	Extended Hückel guess. (**default**)

Note

Reading saved vectors is the standard way to restart a stalled SCF, and to reuse converged orbitals in a follow-up job such as a Hessian after an optimization.

See also: *save ($control)*, *iterations ($scf)*

5.4.2 na

Type: indexed-array
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t74e; 08 t75b; 08 t75d

Default: `na(1)=1,2,3,...,N` (the natural order).

Reordering of the starting vectors for alpha spin.

Note

Accepted in version 25, also together with `read=1`, where it reorders the vectors read from the file. For closed-shell water, swapping the HOMO and LUMO only slowed the SCF: it returned to the ground state. The effect was seen only in the iteration counts, as the output does not print the reordering.

Example:

```
$guess na(25)=26,25 $end    # swap the 25th and 26th vectors
```

See also: *nb ($guess)*

5.4.3 nb

Type: indexed-array
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t74e; 08 t75b

Default: `nb(1)=1,2,3,...,N`.

Reordering of the starting vectors for beta spin.

Note

Accepted in version 25 together with `na`; as for `na`, the output does not print the reordering.

See also: *na ($guess)*

5.4.4 mix

Type: switch
Default: `0`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt (annotation corrupted); 03 #0, §7; 06 §3 (default 0); 08 t60, t73, t74a-d, t79, round 14

Breaks the equivalence of the alpha and beta electrons in the highest occupied orbital. It is meant for open-shell singlets (for example singlet biradicals) and is only sensible with `$scf restrict=0`.

Value	Meaning
`0`	No mixing. (**default**)
`1`	Mix the HOMO and LUMO to break alpha/beta equivalence.

Note

Users report that with non-RI DFT, `mix=1` can converge back to the restricted solution (same energy as RKS). One user saw a lower unrestricted energy with RI-PBE and RI-BLYP, but only by about $3{\cdot}10^{-6}$ hartree, which is close to the SCF convergence threshold. In version 25, `mix=1` with `restrict=0` returned to the restricted solution for singlet O2 (PBE and RI-PBE1) and for Cr2 (PBE, default and BFGS procedures): no broken-symmetry solution was found in any test. Stretched H2 (2.5 Å) with UHF or UKS also stayed restricted with `mix=1`, `2`, `-1`, `10`, `45` or `90`, with or without `read=1`, although

the triplet lies 0.13 hartree lower. So in version 25 `mix` is accepted (integers only; `mix=0.5` is an input error) but had no visible effect. Starting the singlet from saved triplet vectors (`read=1`) did reach a broken-symmetry singlet for O2, 0.048 hartree below the restricted one; it needed more than the default 32 iterations (`iterations=300`). This did not work for H2.

See also: *restrict ($scf)*

5.5 $scf

Spin restriction, convergence, iteration limits and SCF solver.

5.5.1 restrict

Type: switch
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t48

Default: `1` for a closed-shell singlet, otherwise `0`. Version 25 ran O2 with `mult=3` (an even number of electrons) unrestricted without being told.

Spin restriction.

Value	Meaning
0	Spin-unrestricted.
1	Spin-restricted (available only for a few methods).

See also: *mix ($guess)*

5.5.2 convergence

Type: list
Default: `1e-6,1e-3`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §4.5 (output echo, energy gradient); 05 §5.1; 04 #322-#324; 08 t01, t07a, t07b; 08 t22a-c; 08 t74b (gradient vs threshold); 08 Slurm (erations); 08 review check (convergence value series)

Default: the first value is for SCF and the second for CP-SCF.

SCF and CP-SCF convergence thresholds. The SCF value is a threshold on the energy gradient: in version 25, an SCF whose gradient stayed near $2{\cdot}10^{-6}$ ended with `no convergence!` at the default 1e-6. The defaults are recommended. To save time in difficult cases you may loosen the SCF value to `1e-5` for preliminary geometry optimizations.

Note

Put `iterations=` before `convergence=`, or give `convergence` two values. A one-value `convergence=` followed by `iterations=` stops the run with `*** error in '$scf' : unknown name terations`, also when the two keywords are on separate lines. The message names `iterations` with its first letters missing; how many depended on the machine, not on the value (`terations` on one computer, `erations` with `p4_d` on another, for every value tried). Other keywords after a one-value `convergence=`, such

as `restrict=`, were read correctly. `iterations=250 convergence=1e-7` and `convergence=1e-7,1e-3 iterations=250` both work. Check the `scf: conv = ...; iter=...` echo line in the output.

See also: *iterations ($scf)*, *procedure ($scf)*

5.5.3 iterations

Type: list
Default: `32,16`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §4.5; 04 #319; 05 §5; Dft.rtf; 02 §3.2 (pri help); 04 #306; 08 t01, t10, t28, t71b

Default: Version 25 echoes `iter=32,16` for DFT. v6, v13 and v15 echoed `16,16`, a HF run for CCSD showed `48,16`, and `Dft.rtf` gives 30 (older version). The default may depend on the method and version.

Maximum number of iterations and of microiterations (`iterations=it,micro`).

Note

An unconverged SCF does not stop the run. Version 25 prints `no convergence!`, then still prints `E =`, the orbitals and the properties, and exits with status 0. Search the output for `no convergence!`.

Raising the limit moderately is fine: an OH radical failed at the default 32 iterations and converged after 34 with `iterations=100`. A very large limit does not replace understanding why the SCF is not converging (`Dft.rtf` warns against it). A recommended technique for difficult cases is to cap the run at `iterations=10`, save the vectors (`$control save=`) and restart with `$guess read=1`, repeating as needed.

See also: *convergence ($scf)*, *read ($guess)*

5.5.4 procedure

Type: enum
Default: `nr`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf; 04 #320; 08 t07b

Default: `nr` for GGA DFT.

Algorithm for solving the SCF equations. If one procedure fails, try the other.

Value	Meaning
`nr`	Newton–Raphson quadratically convergent. Can recover even from a wrong starting electronic configuration but may blow up in hard cases. (**default**)
`bfgs`	BFGS Hessian update. Less prone to blowing up but usually slower and sometimes converges poorly.

Example:

```
$scf convergence=1e-7 procedure=bfgs iterations=250 $end
$grid accuracy=1e-9 $end
```

5.5.5 d1small

Type: switch
Default: 0
Status: current · not verified
Sources: input.txt

Small-component density in quadratically convergent SCF.

Value	Meaning
1	Include the small-component density in the linear equations of quadratically convergent SCF (scalar-relativistic pure DFT).
0	Discard these contributions. This is faster, but SCF may fail to converge for molecules containing the heaviest atoms. (**default**)

See also: *four ($control)*

5.5.6 core

Type: integer
Default: *none stated*
Status: current · not verified
Sources: input.txt

Default: computed from the atom table; see `$atoms core`.

Number of core electrons for HF-based methods.

See also: *core ($atoms)*

5.6 $dft

Exchange–correlation functional.

5.6.1 functional

Type: enum
Default: PBE
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §3.2; 03 §5 (strings); 04 §0, #347 (no D3); 08 t02; 08 t31, t38

Default: PBE (the default GGA); PBE1 for `theory=ridft`.

Exchange–correlation approximation for DFT methods.

Value	Meaning
PBE	Perdew–Burke–Ernzerhof, Phys. Rev. Lett. 77 (1996) 3865. (**default**)
mPBE	Adamo–Barone modification of PBE, J. Chem. Phys. 116 (2002) 5933.
BLYP	Becke–Lee–Yang–Parr, Phys. Rev. A 38 (1988) 3098; Phys. Rev. B 37 (1988) 785.
OLYP	Handy–Cohen exchange + LYP correlation, J. Chem. Phys. 117 (2002) 1441.
PBE1	PBE hybrid with 1/4 exact exchange (PBE0), J. Chem. Phys. 110 (1999) 6158. Default for `theory=ridft`.
B3LYP_PW	Becke three-parameter hybrid with PW local correlation (see notes).
B3LYP	Becke three-parameter hybrid (see notes).
LPBEVV	Newer functional used in the official example inputs with `theory=DFT_n4` (2025). Runs in version 25; its definition is not documented. *(undocumented)*
LPBEVVV	Newer functional used in the official example inputs with `theory=ridft` (2023–2025). Runs in version 25; its definition is not documented. *(undocumented)*

Note

The B3LYP form in Priroda is Exc = 0.2·Ex(HF) + 0.8·(0.9·Ex(B) + 0.1·Ex(S)) + 0.81·Ec(LYP) + 0.19·Ec(LSD). Ec(LSD) is the local correlation of either Phys. Rev. B 45 (1992) 13244 (PW) or Can. J. Phys. 58 (1980) 1200 (VWN). `input.txt` points out that some commercial programs use VWN, although Becke himself used PW (J. Chem. Phys. 98 (1993) 5648). It does not say which name uses which form. Presumably `B3LYP_PW` uses PW and `B3LYP` uses VWN (inferred from the names; untested). Which VWN parametrization would be used is not stated; `VWN5` appears among the v6 functional names.

Hybrids (`PBE1`, `B3LYP`, `B3LYP_PW`) require `theory=ridft`. With `theory=dft` or `tddft` they either run silently without exact exchange or abort; see `$control theory`.

No empirical dispersion correction (D3 or similar) is available, and a user relayed that none is planned. The names `LPBEVV` and `LPBEVVV` may stand for a VV10-type nonlocal (dispersion) correlation (inferred from the names and the π-stacked examples; unverified). The v6 binary contains further functional names (`VWN5`, `SLYP0`, `BPBE`, `HFLSD`, `HFPBE`, `B2PBE`, `B3PBE`, `HFLYP`, `B1LYP`, `B2LYP`), none of them documented or tested.

See also: *theory ($control)*

5.7 $grid

Adaptive integration grid for DFT.

5.7.1 accuracy

Type: list
Default: `1e-8,1e-5`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §4.4 (output echo); Dft.rtf; 08 t01, t04

Default: `1e-8` for the final grid and `1e-5` for the coarse grid used in the first SCF iterations, as echoed by v13, v15 and v25 (`grid: accuracy=1.0e-08,1.0e-05`). `Dft.rtf` gives `1e-7` (older version).

Target accuracy of the exchange–correlation energy per atom for the adaptively generated grid (DFT methods). The default, 1e-8, is a fine grid. `input.txt` suggests 1e-7 to speed up geometry optimizations with the default `$optimize tolerance=1e-4` or looser, and 1e-9 for very accurate calculations. The grid grows during the SCF; the output reports each size on a line such as `grid:  34000 53` (number of points) in the SCF table.

Note

`Dft.rtf` suggests 1e-6 for rough optimizations (`tol=1e-3`) and 1e-8 for floppy molecules optimized to `tol=1e-5`. It says that asking for more than 1e-8 "makes no deep sense", because the integration formulas cannot reach unlimited accuracy. This conflicts with `input.txt`'s 1e-9 for very accurate work.

See also: *tolerance ($optimize)*

5.8 $integral

Integral evaluation for RI methods.

5.8.1 direct

Type: switch
Default: 0
Status: undocumented · not verified
Sources: input.txt (annotation corrupted); 03 #0, §3, §17; 05 §4.2

Selects direct (recomputed) Coulomb integrals for RI methods.

Value	Meaning
0	Store integrals. (**default**)
1	Recompute Coulomb integrals (direct mode) in `theory=rimp2` and `theory=ridft`.

Note

Direct mode roughly halves disk use, but it is slower. It is also the reported workaround when a small RI-MP2 job claims not to fit in memory, or crashes with `write VMNN(...)?`.

Example:

```
$integral direct=1 rout=2 $end
```

5.8.2 routine

Type: integer
Default: 0
Status: undocumented · not verified
Sources: input.txt (annotation corrupted); 03 #0

Algorithm for the two-electron integrals in correlated methods. `input.txt`'s example uses `rout=2`, but no source explains the value 2.

Value	Meaning
0	Default algorithm. (**default**)
1	Slightly slower but parallelizes better over many processes.

5.9 $atoms

Per-element core definitions for correlated (HF-based) methods.

5.9.1 core

Type: indexed-array
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t16; 01 §7; 08 (review check)

For HF-based methods, the number of core shells for L=0,1,2,3, given for each atomic number. The values are used only to compute the total number of core electrons in the molecule.

Note

The index is 10·Z: `core(280)` is nickel, `core(80)` oxygen. In version 25, `core(80)=0,0,0,0` removed the oxygen 1s from the frozen core of an RI-MP2 run. The output reports the frozen core as `atom 8 core = 1` and `Number of core electrons 1`; the number there apparently counts core orbitals, not electrons (the C60 example output prints `Number of core electrons 60` for 60 carbon atoms, and a version 25 HCl run prints 5 for the Cl core 1s, 2s, 2p).

Example:

```
$atoms core(280)=2,1,0,0 $end
# 1s, 2s, 2p (10 electrons) of nickel are core
```

See also: *core ($scf)*, *mcore ($atoms)*

5.9.2 mcore

Type: integer
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 (review check)

A simple way to add outer-core shells to the correlation treatment: `mcore=M` correlates every core shell whose principal quantum number is at least n(valence) − M. In version 25, for Cl in an RI-MP2 run of HCl, the default core was `atom 17 core = 2, 1` (1s, 2s, 2p); `mcore=1` left only 1s in the core (`core = 1`), and `mcore=2` left no core at all.

See also: *core ($atoms)*

5.10 $optimize

Geometry optimization, saddle-point search, IRC and relaxed scans.

Relevant when: `task = optimize | optimize+hessian | irc | scan | opt_n1`

5.10.1 saddle

Type: switch
Default: 0
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf; 04 #216; 03 §9; 08 t25

Type of stationary point sought.

Value	Meaning
0	Optimize to a minimum. (**default**)
1	Optimize to a saddle point.

Note

A saddle-point search needs an accurate Hessian in the input: compute one with `task=hessian` and append its `$Energy` block (the `eng>` lines). A usual route is scan → Hessian at the scan maximum → `saddle=1` → `task=hessian` to confirm one imaginary frequency → IRC. In version 25 the output confirms the search with `Saddle point search: follow = 1` and, when a Hessian was read from `$Energy`, `Lowest eigenvalues:` with one negative value.

See also: *follow ($optimize)*, *H ($Energy)*

5.10.2 tolerance

Type: list
Default: 0.0001,0.01
Units: au
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §4.6 (output echo); 08 t11, t12

Default: if only the first value is given, the second is 100 times larger (the recommended setting).

Convergence tolerance on the gradient (first value) and on the displacement (second value).

See also: *accuracy ($grid)*

5.10.3 trust

Type: list
Default: 0.125,0.015625
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §4.6; 04 #319; Dft.rtf; 08 t11, t12, t15, t76b; 08 (review check)

Default: echoed by version 25 as `Trust radius = 0.12500000 (max), 0.01562500 (min)` in internal coordinates, with symmetry (`sym=1`) and with `cartesian=1`. v13 and v15 echoed 0.2 / 0.0125 in internal coordinates and 0.1 / 0.00625 with `cartesian=1`; v6 echoed 0.2 / 0.0125 in Cartesian coordinates. `Dft.rtf` gives 0.5 (older version). `input.txt` says that if only the maximum is given, the minimum is 1/16 of it and the initial value 1/4. Version 25 uses 1/8 instead: `trust=0.2` alone was echoed as `0.20000000 (max), 0.02500000 (min)`.

Trust radius (the largest allowed step), given as maximum, minimum and initial values.

5.10.4 follow

Type: integer
Default: 1
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf; 08 t25

Default: 1 according to Dft.rtf; version 25 echoes Saddle point search: follow = 1.

Follow the n-th Hessian eigenvector in a saddle-point search.

See also: *saddle ($optimize)*

5.10.5 steps

Type: integer
Default: 128
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf; 03 §8 (step/steps bug); 08 t14, t15

Maximum number of optimization cycles.

Note

Version 25 echoes Maximum number of steps = 128 when steps is not given; older sources give 50. When the limit is reached the program prints no message: the output just ends, and the last geometry appears only in mol> lines, not MOL>. Dft.rtf warns that an optimization not converged after about 50 steps very likely has something wrong with it, and that computing a Hessian first helps in many cases; so don't simply raise steps. In v6, if both steps and step are given, the value of step is used; give only one of them.

See also: *tolerance ($optimize)*, *steps ($d2edr2)*

5.10.6 cartesian

Type: switch
Default: 0
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 01 §4.6; 04 #518; 08 t11; 08 t76a-d

Coordinate system for the optimization. The output reports Coordinates: internal or cartesian.

Value	Meaning
0	Redundant internal coordinates. (**default**)
1	Cartesian coordinates (last resort).

Note

cartesian=1 is the fix when the program crashes with process 0 got signal 6 after a too few message in the internal-coordinate table. In version 25, bent CO2 and acetylene that became linear during the optimization converged in internal coordinates without that crash (the number of internal coordinates dropped instead), and also with cartesian=1.

5.10.7 back

Type: switch
Default: 0
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 02 §3.2; 04 §0; 08 t26, t42

IRC direction. A full IRC takes two runs, one per direction: version 25 with the default `back=0` followed one direction only (`n = 1, back = 0, down = 1` in the output), and `back=1` the other one (negative `distance` values on the `IRC>` lines). Re-optimize the endpoints afterwards, because an IRC does not end at a converged minimum.

Value	Meaning
0	Trace the IRC path forwards. (**default**)
1	Trace the IRC path backwards.

See also: *points ($optimize)*, *radius ($optimize)*

5.10.8 points

Type: integer
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t26, t37

Number of points to optimize in an IRC run, or number of constrained geometry optimizations in a scan run. In a scan the points include both ends: `points=4` gives s = 0, 1/3, 2/3 and 1. An IRC can stop before `points` is reached when it arrives at a minimum.

See also: *value ($optimize)*

5.10.9 radius

Type: real
Default: 0.1
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf; 04 #298; 08 t26; 08 t77d-h (t77h: back=0 only)

Default: `0.1` as echoed by version 25 (`r = 0.1000000000`); `Dft.rtf` gives 0.2 (older version).

Radius, in mass-weighted coordinates, for IRC path optimization. Adjacent IRC points end up roughly twice this distance apart. The IRC coordinate is reported in bohr·amuˆ1/2.

Note

`Vicinity of a Stationary Point(?) --- STOP.` ends an IRC that has reached the region of a minimum; in version 25 it was the normal end of a test IRC. Users have also seen it early, when the energy did not decrease on a step; changing the step size is the reported fix there. The other normal end is `Energy Minimum Passed --- STOP.`, printed after a point that came out higher than the one before; in version 25 it sometimes followed the `Vicinity` message. In a version 25 IRC of HCN ⇄ HNC, `radius=0.03` and `0.1` reached both minima (about 65 and 20 points per direction). `radius=0.3` was run towards HCN only: 7 points, stopping farther from the minimum than the smaller radii.

5.10.10 coordinates

Type: list
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 03 §5 (type 4); 08 t40b

Default: generated automatically.

List of redundant internal coordinates to use, each given as five integers.

Value	Meaning
`1,i,j,0,0`	Distance i–j.
`2,i,j,k,0`	Angle i–j–k.
`3,i,j,k,l`	Torsion i–j–k–l.
`4,i,j,k,l`	Special coordinate for linear fragments. A user describes it as the angle between line l–k and the normal to plane k–j–i.
`5,i,j,k,l`	Special coordinate for linear fragments (meaning not described).

Note

`input.txt` spells this keyword `coordinated=`, which is a typo: version 25 stops with `*** error in '$optimize' : unknown name coordinated`. Use `coordinates=`.

See also: *fix ($optimize)*

5.10.11 fix

Type: list
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 02 §3.2; 03 §5 (2010 wish list); 04 #372-#373; 08 t37; 09

Internal coordinates to keep fixed during the optimization, in the same five-integer format as `coordinates`. Several entries are separated by commas or spaces.

Note

Users reported in 2010 that atoms could not be frozen in Cartesian space and that two coordinates could not be constrained to stay equal. This was not checked in version 25, whose executable contains `$optimize` names of unknown meaning (`fragment`, `weight`, `local`) and restraint groups (`$anchor`, `$confine`).

See also: *value ($optimize)*, *coordinates ($optimize)*

5.10.12 value

Type: indexed-array
Default: *none stated*
Units: Å and degrees
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 03 §5, §8; 04 #278-#279, #373; 08 t37

Default: computed from the input geometry.

Values of the fixed internal coordinates. For `task=scan`, give all the initial values followed by all the final values. `val(i)=` fills the list starting at position i, so values left unset keep their input-geometry values.

Note

Priroda's dihedral angles have the opposite sign to the convention used by Molden, MOPAC and GAMESS. Flip the sign of any torsion value taken from another program, or the scan starts from a mangled geometry.

Example:

```
$optimize
 fix   =1,7,9,0,0, 1,8,10,0,0, 3,8,7,9,0, 3,10,9,7,0
 val(1)=1.6,       1.7
 val(5)=2.0,       2.1
 points=5
$end
# 5 optimizations: r(7-9) = 1.6 ... 2.0, r(8-10) = 1.7 ... 2.1;
# the last two fixed coordinates keep their input values
```

See also: *fix ($optimize)*, *points ($optimize)*

5.10.13 gradient

Type: switch
Default: *none stated*
Status: current · not verified
Sources: input.txt

Set `gradient=0` when the starting second derivatives were computed with a different method or basis set. This excludes the gradient terms from the internal/Cartesian transformation at the first geometry. Otherwise keep the default.

5.10.14 print

Type: switch
Default: *none stated*
Status: current · not verified
Sources: input.txt

Amount of optimization output.

Value	Meaning
0	Minimal.
1	Detailed

5.10.15 symmetry

Type: switch
Default: *none stated*
Status: undocumented · verified on 25 (2025.02.28)
Sources: 01 §4.6; 04 #481; 09; 08 t15; 08 t30b

Keeps point-group symmetry during the optimization (the optimizer switches to Cartesian coordinates); needs `$molecule sym=` as well. The official 2023–2025 example inputs write it as the prefix `sym=1`. In version 25, together with `$molecule sym=1,0.01`, the optimizer switched to Cartesian coordinates (`Coordinates: cartesian`), printed `symmetry: C3v (6)` at every step, and returned an exactly symmetric structure followed by `MOL> symmetry: C3v`. Without `$molecule sym=`, `$optimize sym=1` only switches to Cartesian coordinates and reports `symmetry: C1 (1)`, so give both options.

See also: *sym ($molecule)*

5.10.16 model

Type: integer
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.6; 09

Used as `mod=1` with `task=opt_n1` (very large systems). In version 25, `model` is read only by the separate optimizer of `task=opt_n1`, together with `function`, `h0`, `delta`, `fragment`, `steps`, `trust`, `tolerance` and `print`. Meaning not documented.

5.11 $d2edr2

Numerical Hessian by finite differences of the gradient.

Relevant when: `numerical Hessian: task = hessian with theory = ridft | rimp2 | tddft | qm_n3, or $control derivative = numeric (an analytic dft Hessian has no displacement steps)`

5.11.1 length

Type: real
Default: `0.001`
Units: au (bohr)
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t32a, t55 (raw output)

Default: version 25 prints `Step 0.00100000 au FORWARD for coordinate 1:` for each displacement.

Step length for numerical differentiation of the potential-energy surface.

5.11.2 displace

Type: switch
Default: `2`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t32a; 08 t55

Number of displacements per coordinate. More accurate numerical Hessians may also need tighter SCF convergence.

Value	Meaning
1	One displacement per coordinate.
2	Two displacements per coordinate. More accurate but twice as slow. (**default**)

Note

With the default, version 25 printed a FORWARD and a BACKWARD step for every coordinate.

See also: *convergence ($scf)*

5.11.3 steps

Type: list
Default: `0,-1`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t55; 08 t32a, t32b, t53, t65

Default: `0,-1` runs every step from 0 to 3N (or 6N) in a single run.

The first and last displacement steps to do in this run. The reference geometry is step 0, and the displaced geometries are 1 … 3N (one displacement) or 1 … 6N (two displacements). This lets a numerical Hessian be split across separate runs and then combined.

Note

Not the same keyword as `$optimize steps`. Tested in version 25 with water (`ridft`, `displace=1`, steps 0–9): two partial runs (`steps=0,4`, `steps=5,9`) and a combining run with `displace=1 steps=-1` gave the same frequencies, to within 0.4 cm^{-1}, as a one-run Hessian with the default `displace=2`. The combining run does no SCF. Each part writes one numbered group per displacement (`$Energy000`, `$Energy001`, …) on `num>` lines. The collecting command in `input.txt` (`grep "num>" ... | sed "s/\.*num>//g"`) leaves the file names that `grep` adds at the start of each line; the example uses `grep -h` instead.

Example:

```
# 12-atom molecule, four inputs run in parallel on different machines:
$d2edr2 disp=1 steps= 0, 9 $end    # mol.in1
$d2edr2 disp=1 steps=10,18 $end    # mol.in2
$d2edr2 disp=1 steps=19,27 $end    # mol.in3
$d2edr2 disp=1 steps=28,36 $end    # mol.in4
# collect the partial results into one input:
grep -h "^num>" mol.out1 mol.out2 mol.out3 mol.out4 | sed 's/^num>//' >> mol.in
# in mol.in, replace the $d2edr2 line with the one below and run once more to process␣
↪them:
$d2edr2 disp=1 steps=-1 $end
```

See also: *E ($Energy)*

5.12 $thermo

Thermochemistry after a Hessian calculation.

5.12.1 sigma

Type: integer
Default: `1`
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 08 t13; 08 t44

Default: 1 (version 25 does not detect the symmetry number).

Rotational symmetry number for thermochemistry. Version 25 does not detect it and uses 1 unless `sigma=` is given, which overestimates the rotational entropy by R ln σ (2.18 cal/(mol·K) for NH3, $\sigma = 3$). With `sigma=3`, NH3 gave S = 46.03 cal/(mol·K) at 298.15 K.

5.12.2 `temperature`

Type: list
Default: `298.15`
Units: K
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; Dft.rtf; 08 t13; 08 t44; 04 #80, #476

Default: `298.15`: version 25 prints `T = 298.150 K`, as `Dft.rtf` states. Older posted outputs also show `298.000 K` (2011, 2021), so set `temperature=` explicitly when comparing with other versions.

Temperatures at which thermochemistry is printed, up to 50 values. Each temperature gets its own table: `temperature=200,298.15,400` printed three.

5.12.3 `mass`

Type: list
Default: *none stated*
Units: amu
Status: undocumented · not verified
Sources: Dft.rtf

Atomic masses for vibrational analysis and thermochemistry. Use it to redo the analysis for other isotopes from a stored Hessian (`$Energy`) without recomputing it.

See also: *mass ($molecule)*, *H ($Energy)*

5.13 `$nmr`

Reference shieldings and extra points for NMR shielding calculations.

Relevant when: `task = nmr (theory = dft only)`

5.13.1 `standard`

Type: indexed-array
Default: *none stated*
Status: current · verified on 25 (2025.02.28)
Sources: input.txt; 04 #69; 02 §5.7; 08 t34; 08 t57b

Reference shielding constants, indexed by atomic number, for computing chemical shifts. Values are needed only for the elements present in the molecule.

Note

A reference shielding is only valid for the method, basis set and grid it was computed with. A user input posted on a forum has other TMS values (`standard(1)=31.3867 standard(6)=181.9781`); the settings they were computed with are not given. Compute TMS with exactly the settings of the production run. Without `standard`, version 25 prints `shift=` as minus the isotropic shielding; with it, `shift = standard − isotropic shielding`.

Example:

```
$nmr
 standard(1)=31.3915
 standard(6)=181.9921
$end
# computed for Si(CH3)4; usable for a series of hydrocarbons
```

5.13.2 points

Type: list
Default: *none stated*
Units: Å
Status: current · not verified
Sources: input.txt; 02 §3.2; 03 §12

Extra points in space, given as x,y,z triples, at which magnetic shielding tensors are computed in addition to the nuclear centres. This is the way to compute NICS: NICS is the negative of the isotropic shielding printed for the point.

Example:

```
$nmr
 points=
      0.00000000    0.00000000    0.00000000
      0.00000000    0.00000000    1.00000000
$end
```

5.14 $Energy

Results of an earlier run (energy, gradient, Hessian, dipole data), read back as input.

The program prints this group on lines prefixed with `eng>` (an energy, gradient or Hessian run) or `num>`. `num>` lines come from `task=dipole` (one `$Energy` group with `A=`, `Dx=` and `Ax=`) and from numerical Hessians, which write one numbered group per displacement: `$Energy000`, `$Energy001`, ... (see `$d2edr2 steps`). Strip the prefix and append the groups to the next input. The helper script `ge` on the official site does this.

A stored Hessian is required for `$optimize saddle=1` and `task=irc`. It also speeds up difficult minimizations, and it lets you redo vibrational analysis or thermochemistry (for example for other isotopes, `$thermo mass=`) without recomputing.

```
eng>$Energy
eng> E=-2.284057276940e+03
eng> D= 4.22290114e-08 -1.82061636e-08  2.32661117e-08
eng> G= 5.13271929e-06 -5.52234429e-06 -6.15997899e-06
eng>    ...
eng>$end
```

5.14.1 E

Type: real
Default: *none stated*
Units: hartree
Status: undocumented · verified on 25 (2025.02.28)
Sources: 01 §4.10; 02 §3.2; 08 t01

Total energy.

5.14.2 G

Type: list
Default: *none stated*
Units: au
Status: undocumented · verified on 25 (2025.02.28)
Sources: 01 §4.10; 02 §3.2; 08 t02

Cartesian gradient (3N values).

5.14.3 H

Type: list
Default: *none stated*
Units: au
Status: undocumented · verified on 25 (2025.02.28)
Sources: 02 §3.2; 08 t13; 08 t24, t25, t41, t41b; 08 t55

Hessian as a lower triangle, 3N(3N+1)/2 values. Written by `task=hessian` on `eng>` lines; in version 25 a `saddle=1` optimization and an IRC read it back from an appended `$Energy` group. If the input holds several plain `$Energy` groups, the last one is used. The numbered groups `$Energy000`, `$Energy001`, ... of a split numerical Hessian are all read by the combining run (`$d2edr2 steps=-1`).

See also: *saddle ($optimize)*, *steps ($d2edr2)*

5.14.4 D

Type: list
Default: *none stated*
Units: au
Status: undocumented · verified on 25 (2025.02.28)
Sources: 01 §4.10; 02 §3.2; 08 t01

Dipole moment (3 values), written next to `E=`.

5.14.5 Dx

Type: list
Default: *none stated*
Status: undocumented · not verified
Sources: 02 §3.2 (inferred); 08 t13; 08 t36

Dipole derivatives with respect to nuclear coordinates (inferred from use: IR/Raman intensities).

5.14.6 A

Type: list
Default: *none stated*
Status: undocumented · not verified
Sources: 02 §3.2; 08 t13; 08 t36

Static dipole polarizability as a lower triangle (6 values), written with a negative sign (inferred: the second derivative of the energy with respect to the field). Written by an analytic Hessian (`eng>`) and by `task=dipole` (`num>`).

See also: *Ax ($Energy)*

5.14.7 Ax

Type: list
Default: *none stated*
Status: undocumented · verified on 25 (2025.02.28)
Sources: 08 t36; 08 t56; 04 #381 (relayed); 09

Polarizability derivatives with respect to the nuclear coordinates. Written by `task=dipole` on `num>` lines; appended to a Hessian input, they give the Raman activities and depolarization ratios.

See also: *A ($Energy)*, *Dx ($Energy)*

5.15 $dynamic

Molecular dynamics with a thermostat.

Relevant when: `task = thermostat`

Known from the official example inputs, a third-party input generator, the names in the version 25 executable and one version 25 test. Only the meaning of `n` and `file` is shown by output; the rest is inferred.

In version 25, `task=thermostat` with `e=0.001 d=0.03 n=20,5 m=10 i=1` ran with every tested theory (`dft`, `ridft`, `rimp2`, `ccsd`, `tddft`, `qm_n3`). The header echoes `e = 0.00100000` and `dx = 0.03000000 L = 20, -5;10`. The output has one line per step, `point N: s ...  E ...  g ...`, where negative point numbers are equilibration steps (the second value of `n`) and positive ones the production steps (the first value). The official example (below) writes `d=` and the generator script writes `h=` and `v=`; these are prefixes of the full names `dx`, `hb` and `v1`.

```
$dynamic
 e=0.001
 d=0.03
 n=1000,100
 m=10
 i=1
 file=H40C20/H40C20_%06d.qm.in
$end
```

5.15.1 e

Type: real
Default: *none stated*
Units: hartree
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2; 08 t69; 09

Thermostat temperature as kT (inferred: a generator script sets e = T/315775). Version 25 echoes it as `e = 0.00100000`.

5.15.2 dx

Type: real
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2; 08 t69; 09

Time step (inferred). The examples write the prefix `d=`; version 25 echoes it as `dx = 0.03000000`.

5.15.3 n

Type: list
Default: *none stated*
Status: undocumented · verified on 25 (2025.02.28)
Sources: 01 §4.8; 02 §3.2; 08 t69; 09

Two values: the number of production steps, then the number of equilibration steps. In version 25, `n=20,5` gave 26 points, `point -5 ... point 20`, and the echo `L = 20,-5;10`.

5.15.4 m

Type: integer
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2; 08 t69; 09

Meaning not known. With `m=10`, version 25 printed `;10` at the end of the echo `L = 20,-5;10`.

5.15.5 i

Type: integer
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2; 08 t69; 09

Run index or random seed (inferred). Version 25 does not echo it.

5.15.6 v1

Type: real
Default: *none stated*
Units: bohr^3 per atom
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2; 09

Volume per atom of a spherical confining cavity, e.g. for simulating a liquid (inferred). Written as the prefix `v=` in the generator script.

5.15.7 hb

Type: real
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2; 09

Stiffness of the cavity wall (inferred). Written as the prefix `h=` in the generator script.

5.15.8 file

Type: string
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8

printf-style pattern for snapshot files, which are written as ready-to-run inputs; `%06d` is the frame number.

5.16 $uniq

Duplicate-structure filtering over several `$molecule` blocks.

Relevant when: `task = uniq`

In version 25, `task=uniq` with `dist=0.01` and several `$molecule` blocks in one input did no SCF. It listed each structure as new (`1 => 1`) or as a copy of an earlier one with a distance (`2 = 1 0.0060421693`), printed `2 unique`, and wrote the `Atomic Coordinates` of each unique structure. A copy of a water molecule that was rotated and translated counted as identical (distance 0), and so did a copy with one H moved by 0.004 Å; a copy with one H moved by 0.043 Å counted as a new structure.

5.16.1 distance

Type: real
Default: *none stated*
Status: undocumented · verified on 25 (2025.02.28)
Sources: 01 §4.9; 09; 08 (review check)

Threshold for treating two structures as identical: structures closer than this count as copies. The official example uses `dist=0.01` on 157 MD snapshots, each in its own `$molecule` block. How the distance is measured and in which units is not known. `dist` is a prefix of the full name `distance`; the group also reads `file`.

CHAPTER

SIX

SOLVING COMMON PROBLEMS

This chapter is arranged by symptom. Each entry names the likely cause and the fix that users have reported.

Almost everything here comes from user forums, not from the developer. Each fix says how well it is supported: "confirmed by several users" means more than one person reported that the fix worked, and "single report" means one user posted it and nobody confirmed it. Most reports date from Priroda 6 to 10 (about 2009–2015); a few are from 2008 or from 2016–2026. Where a problem may have been fixed in later builds, the entry says so.

Note

Most fixes below have not been checked against a current Priroda binary. Where a test run on version 25 (2025.02.28) confirmed or contradicted a report, the entry says so.

6.1 The input is not read, or is read wrongly

When Priroda cannot parse the input it often stops early, with little or no explanation. Just as often a mistake is silently ignored and the run goes on: in version 25 a group written as `&dft` was skipped, an unknown `theory` ran Hartree–Fock, an unknown `task` only set up the job, keywords in a group the task does not read were not checked, and atomic numbers written as `8.0` gave a wrong molecule (see below). Before anything else, open the output and check what it echoed back: the `Memory:` and `Disk:` lines, the `formula:` line, the summary box (number of atoms, charge, multiplicity, method, functional), the `scf:` line and the `grid:` line. If a value you set is not shown there, Priroda did not read it.

6.1.1 Output stops after `molecule input: ''`

The line `molecule input: ''` itself is printed in every run. It is a problem only when the output ends there or continues with garbled text.

Cause. In almost every reported case the input file had Windows (CRLF) line endings. The tail of the output then shows a mangled keyword followed by `?`, for example `' ?matrix`, `?rtesian` or `'z-matrix' ?`. Inputs saved as Unicode, or edited in Word, were also reported not to be read. Word's "save as text" still writes CRLF line endings.

Fix. Save the input as plain ASCII text with Unix (LF) line endings, for example with `dos2unix input.in`. Notepad++ and Far Manager (Shift+F2 in the editor) can also convert line endings. Use straight quotes only. Several users confirmed this fix. Running `dos2unix` on a file that already has LF endings does no harm.

6.1.2 The molecule is wrong: odd formula, extra atoms

Atomic numbers written as real numbers (`8.0` instead of `8`) in `$molecule`. The atom lines are read as a stream of numbers, so everything after the first `.0` shifts. In version 25 there is **no error**: water written with `8.0` and `1.0` ran as four atoms, an oxygen and three empty centres with Z=0, with the formula `__3O` and a meaningless energy. An older version was reported to stop with `illegal charge/multiplicity` instead. Write atomic numbers as integers, and check the `formula:` line and the atom count at the top of every output.

One experienced user also advises avoiding Cyrillic letters and spaces in the computer name, directory names and file names. In the case where he gave this advice, fixing the line endings alone was reported to solve the problem.

6.1.3 `task=scan` runs an optimization, or nothing works on one machine

An input that runs on one computer but not on another, or a `task=scan` that silently runs a plain optimization, has been traced to the same encoding problem: an input edited in Word or saved as Unicode. Fix it as above.

Two `task=` lines in one input do not chain jobs, and no error is printed. In version 25 only the last one ran (`task=optimize` then `task=hessian` gave just a Hessian); in a 2011 report (version 6) only the first one ran. The only combined task name is `task=optimize+hessian`; see *task*. A user in 2011 added that even this one did not work in all versions. Run other sequences, such as optimization followed by NMR, as separate jobs.

6.1.4 The basis file is not found

- Quote file names with straight double quotes: `basis="3z.bas"`. Two single quotes (`basis=''3z.bas''`) give `error opening file ''3z.bas''`, and typographic quotes copied from a web page or word processor fail too.
- Give an absolute path to the basis file if Priroda is not started from the directory that contains it. One user's parallel run aborted with `MPI_Abort` until he wrote the full basis path in the input. In version 25 a relative name worked with the file in the current directory, in a serial and in a 2-process run.
- The relativistic basis file `basis4.in` needs *four=1* in *$control*. Without it the job stops right after the `Basis set input:` line, with `1?` as the only message (version 25). The user who first reported this confirmed the fix.
- The name given in *set* must exist in the basis file.

6.1.5 The input or output file cannot be opened

On Linux the usual cause is a wrong path. Check it with `ls -l` before blaming Priroda. In one reported case the directory in the command line simply did not exist. Work in a directory you own and can write to (for example `~/priroda`), `cd` into it, and run the binary with an explicit path, since the current directory is normally not in `PATH`:

```
cd ~/priroda
./p job.in job.out
```

File names are case-sensitive, unlike keywords. One user of an older version saw the extra complaint `0: output file 'tasK.out'?` when the file name contained capital letters. Version 25 wrote an output file with capital letters in its name without complaint.

On Windows, `cd` into the folder with the input before running `p.exe input.in output.out`. One user still got `cannot open input file` after doing that, and the thread never found the cause. Giving full Windows paths for both files, such as `p1.exe C:\Work\mol.in C:\Work\mol.out`, works from any folder.

6.1.6 Output stops at `0: opening file /tmp/...` (Windows)

Windows builds put scratch files in `/tmp` when nothing else is set (the output echoes `Disk: 1g '/tmp'`), and Windows has no `/tmp`. The current `p1.exe` (version 20) still does this: the output ends with `0: opening file /tmp/tmp978.0000` and `process 0 got signal 6`. The Linux version 25 has the same default and also honours `TMPDIR`. Set the scratch directory explicitly, for example `$system path=. $end`, or set the `TMPDIR` environment variable before starting the program. A RAM disk (negative *disk*) also avoids the problem. This is the oldest known problem in both forums. See *path*.

In `cmd.exe`, write `set TMPDIR=%TEMP%` or `set "TMPDIR=%TEMP%"`. With `set TMPDIR="%TEMP%"` the quotes become part of the value: the `Disk:` echo shows the quoted path appended to the current directory, and the run stops in the same way.

6.1.7 `$scf` settings are ignored or rejected

The parser has a problem when *convergence* is directly followed by another keyword in *$scf*. In version 25,

```
$scf convergence=1e-7 iterations=250 $end
```

stops the run after the basis-set echo with `*** error in '$scf' : unknown name terations`. The message names `iterations` with its first letters missing. How many are missing depended on the machine, not on the value: the same input printed `unknown name erations` with the `p4_d` executable on another computer, for every value tried, as forum users of 2015 builds also reported. With *procedure=bfgs* between them, `$scf convergence=1e-7 procedure=bfgs iterations=250 $end`, both values are read; one forum user found that the pair worked only with `procedure=bfgs`.

Further tests on version 25 narrowed it down. The problem is a `convergence=` with **one** value followed by `iterations=`, also on the next line; other keywords after it, such as `restrict=` or `procedure=`, were read correctly. Both of these work:

```
$scf convergence=1e-7,1e-3 iterations=250 $end
$scf iterations=250 convergence=1e-7 $end
```

So give `iterations` before `convergence`, or give `convergence` two values, and confirm the values in the `scf:` echo line of the output, for example `scf: conv = 1.0e-07, 1.0e-03; iter=250,16`.

6.2 SCF does not converge or diverges

6.2.1 Recognizing the problem

Users report that Priroda's SCF usually converges well, better than in many other programs. Trouble mostly comes with metals in low oxidation states, where the electronic structure has some multiconfigurational character, with very large basis sets, and with diffuse functions.

Signs of trouble in the SCF table:

- `hmin` values that keep growing in magnitude, or a line such as `h = -25.15!`;
- `lowest eigenvalue = ...!!!` lines with large negative numbers;
- an energy that rises from one macro-iteration to the next;
- `no convergence!`, in older versions followed by `SCF is far from convergence`.

Warning

In version 25, a single-point run whose SCF hits the iteration limit prints `no convergence!` and then carries on: it prints an energy, orbitals and properties and exits normally. An OH radical with default settings did this after the default 32 iterations, still creeping towards convergence; with `iterations=100` it converged after 34. Always check for `converged!`.

The DFT SCF runs on a coarse grid first and then switches to the final grid; the switch is marked by a new `grid:` line with the number of points. A short jump in the gradient at that point is normal. Steady growth after it is not (see *Divergence after the switch to the fine grid (open-shell transition-metal complex)*).

6.2.2 Check the input first

In one well-documented failure, several input errors were present besides the SCF trouble. Before changing SCF settings, check:

- **Multiplicity.** *mult* must match the electron count. One failing input gave `mult=0` for an odd number of electrons, and the summary box showed `Spin multiplicity 0`.
- **Theory and functional.** A hybrid functional with `theory=dft` is wrong; see *Hybrid functional silently ignored*.
- **Basis.** Non-native basis sets and diffuse functions converge badly. A non-native basis file must also supply a fitting (auxiliary) basis, and a badly prepared one causes trouble. Use a native basis, or start from a geometry optimized in one.
- **Geometry.** A very poor starting geometry can by itself prevent SCF convergence. Pre-optimize with a cheap pure functional (for example PBE with `theory=dft`) before switching to the target method.

6.2.3 Restart from saved vectors

This technique is recommended on both forums (by amg and by amge, who may be the same person), and the limor1 `pri` driver script automates it:

1. Cap the number of SCF iterations at a moderate value, for example `iterations=10` in *$scf*. Choose the cap so that `hmin` does not run away.
2. Have Priroda write its vectors with *save* in *$control*.
3. When the SCF ends with `no convergence!` (older versions stopped with `SCF is far from convergence`), add `$guess read=1 $end` (see *read*) and rerun with the latest geometry.
4. Repeat until the SCF converges.

```
$control
 task=optimize
 theory=dft
 basis="basis.in"
 save=job.vec
$end
$scf iterations=10 $end
$guess read=1 $end
```

An optimization in version 25 does not stop when the SCF of one step fails to converge: it goes on to the next step, which starts from the last orbitals. In a test on water with `iterations=3`, the first step ended with `no convergence!`, the later steps converged, and the optimization reached the same energy as a normal run in the same number of steps. So the loop above matters mostly for single points and for the first SCF of a hard case; check every step of an optimization for `no convergence!` all the same.

In version 25 the output of a restarted run shows `Initial Guess (-1)` and `vectors from 'job.vec'`; a normal start shows `Initial Guess (2)`. Each restart continues from where the previous run stopped, and goes straight to the final grid instead of starting again on the coarse one. In a test on a 22-atom copper complex with 3 iterations per run, the loop converged in the fourth run, after about 12 iterations in all.

Leave out the `$guess` line in the first run. Vectors from a converged run of an easier case, or of the same molecule at another geometry, also work as a starting point.

6.2.4 Switch the SCF procedure and tighten the grid

The default *procedure* is Newton–Raphson (`nr`). If it blows up, try `bfgs`, and the other way round. One experienced user recommends this setting for difficult cases, "which is nearly all of them":

```
$scf
 convergence=1e-7
 procedure=bfgs
 iterations=250
$end
$grid
 accuracy=1e-9
$end
```

He notes that the default 16 iterations of the versions of that time are too few for many jobs; version 25 defaults to 32. If you tighten *convergence* to `1e-8`, make the grid finer as well, `1e-9` or `1e-10` (see *accuracy*).

Note

The user who asked never reported whether this fixed the failing job. `bfgs` is not a cure-all: in the case in the next section it may have been part of the problem.

6.2.5 Divergence after the switch to the fine grid (open-shell transition-metal complex)

Symptom. A doublet Cu(II) complex (OLYP/L2, `restrict=0`, `procedure=BFGS`, `$grid acc=1e-12`, `task=hessian`) converged on the coarse grid. After the switch to the final grid it oscillated for about 30 iterations and then diverged, with the total energy reaching +29 hartree. BLYP behaved the same way.

Note

The same molecule and SCF settings, run as a single point, converge in version 25: with the original BFGS settings in 48 iterations, and with the default Newton–Raphson procedure in 10 iterations and about half the time. The divergence did not reproduce. If you meet it in a current version, the fixes below are still the ones to try.

Fixes. Two experienced users solved it in two ways, which they considered equivalent:

- Run `task=optimize+hessian` instead of a single-point Hessian, and drop `procedure=BFGS` and `iterations`:

  ```
  $control task=optimize+hessian $end
  $scf restrict=0 convergence=1e-10 $end
  ```

  Each optimization step then starts from the previous wavefunction. In version 25 this route converged the optimization of the complex in 6 steps, with every SCF converging, and the Hessian finished: 32 minutes in all on 8 processes, with one small imaginary frequency (6.11i cm^{-1}).

- Save the wavefunction at the start of the optimization with *save* and read it into the Hessian job with `$guess read=1`. For an already optimized structure, the guess built from scratch turned out to be poor.

6.2.6 Open-shell singlets and broken symmetry

For an open-shell (broken-symmetry) singlet, start the unrestricted singlet from the vectors of the triplet. This is the route that worked in version 25 tests: run the triplet with *save*, then the singlet with `restrict=0` and `$guess read=1` (the program accepts vectors of another multiplicity). For O2 this converged, after many iterations, to a broken-symmetry singlet 0.048 hartree below the closed-shell one and 0.015 hartree above the triplet. Give such runs plenty of *iterations*, and check the result as described below.

This route works only when the triplet's orbitals themselves give the singlet an asymmetric start. For O2 they did, because the two singly occupied π^* orbitals are degenerate. For stretched H2 they did not: started from the triplet's vectors, the singlet fell back to the restricted solution.

Neither the atomic spin populations (`print=+charges`) nor the orbital energies showed that this solution was broken-symmetry: in O2 both came out the same for broken-symmetry and closed-shell solutions, because the two π^* orbitals are spread equally over both atoms. What showed it was the energy of a second run from the same triplet vectors with `restrict=1`, which fell back to the closed-shell energy. Use the same check when the spins do not decide.

The older sources recommend instead an unrestricted calculation with the HOMO and LUMO of the guess mixed:

```
$scf restrict=0 $end
$guess mix=1 $end
```

Warning

In version 25 tests `mix=1` had no visible effect: no run with it broke the symmetry, not even stretched H2 with UHF, the textbook case. Use the triplet-vectors route above, and check the result.

See *restrict* and *mix*. In version 25 tests, `restrict=0 mix=1` converged back to the restricted solution for singlet O2 with `theory=dft` (PBE) and for the Cr2 dimer with both the default and the `bfgs` procedure: the energies equalled the closed-shell ones exactly. With `theory=ridft` (PBE1) the O2 run also gave identical alpha and beta orbital energies, but no closed-shell reference was run, so that case is undecided. The same happened for H2 stretched to 2.5 Å, with both HF and PBE, for every integer value of `mix` tried (1, 2, −1, 10, 45, 90): the singlet stayed at the restricted energy, −0.8632 hartree for HF, although the triplet lies at −0.9972. An unrestricted SCF in Priroda keeps whatever symmetry its starting orbitals have, and `mix=1` did not supply the asymmetry.

In a 2011 forum report with non-RI pure DFT (`theory=dft`, PBE or BLYP), `restrict=0 mix=1` likewise converged back to the restricted solution, with identical energies and identical alpha and beta orbital energies. With RI-DFT the same user got an unrestricted energy about 3 microhartree lower, which he took as broken symmetry. Another user notes that pure GGA functionals often do not break symmetry, and suggests a hybrid (with `theory=ridft`) or a Hartree–Fock calculation first.

6.2.7 LUMO below the HOMO

In the orbital energies printed after the SCF, the lowest virtual orbital normally lies above the highest occupied one. If it lies below, either the SCF has settled in an excited configuration, or the molecule has a partly filled set of degenerate orbitals that a closed-shell calculation cannot fill in order.

The second case happened for singlet O2 in version 25 tests. In a closed-shell (`mult=1`) run the pair occupies

one of the two degenerate π^* orbitals. The occupied orbital ended at -0.178 hartree, above the empty one at -0.219. Starting with the two orbitals swapped (*na*, *nb*; whether the swap took effect is not shown in the output), or from the triplet's vectors with `restrict=1`, gave the same state and energy, since filling the other π^* orbital is equivalent by symmetry. For such molecules the closed-shell energy is not a meaningful singlet energy: use an unrestricted broken-symmetry run (see *Open-shell singlets and broken symmetry*) or the high-spin state.

The first case could not be produced in version 25 tests. Water started with its HOMO and LUMO swapped, either from the default guess or from vectors read from a file, went back to the ground state every time; the swap only made the SCF take two to three times more iterations. In the version 25 tests, the only persistent case was the degenerate shell of O2.

Note

If an excited configuration does turn up in a molecule without degenerate orbitals, two tools are available: start from other vectors (`$guess read=1` with vectors of a related calculation), or reorder the starting orbitals with *na* and *nb*. Neither was tested for this manual on such a case, because none was found.

6.2.8 Heavy elements

For scalar-relativistic pure DFT with the heaviest atoms, the v5 documentation suggests *d1small=1*; without it, the quadratically convergent SCF may fail.

If an orbital ordering problem is suspected, *na* and *nb* reorder the starting vectors.

6.2.9 Cases without a known fix

- An HF reference for CCSD diverged for a small closed-shell molecule (7 atoms, `set=L1`, `restrict=0`). The `change` column stuck at `0.50000000`. Nobody replied. Running a closed-shell molecule with `restrict=0` was unusual and may be worth changing.
- An HfCl2 molecule (PBE, `four=1`, `basis4.in`, `set=L3`, `accuracy=1e-10`) would not converge. Nobody replied. Two things in the posted input are worth checking in such a case (this is the manual's observation, not a tested fix): it had `$guess read=1` in the same job that wrote the vectors with `save=`, so the start depended on whatever vector file already existed; and it used `mult=1` for a d^2 Hf(II) compound.

6.3 Geometry optimization problems

6.3.1 The optimization does not converge within `steps`

The default limit is 128 steps in version 25 and 50 in older versions (see *steps*). When the limit is reached, the output just ends: there is no message and no `MOL>` block. `Dft.rtf`, an older unofficial Russian description, warns that an optimization not converged after about 50 steps very likely has something wrong with it, and that computing a Hessian first helps in many cases. So don't simply raise `steps`. What users do:

- **Restart from the last geometry.** The limor1 `pri` script restarts Priroda from the last geometry whenever an optimization runs out of steps. This helps when the optimizer cycles around one geometry even after 300 steps.
- **Supply a Hessian.** Run `task=hessian`, append its `$Energy` block (the `eng>` lines) to the input, and optimize again. See *$Energy*.
- **Use Cartesian coordinates.** *cartesian=1* is called a last resort on the v5 keyword page, but the limor1 `conformers` script uses it for automated runs, and one experienced user says floppy systems

such as explicit solvation shells have to be optimized in Cartesian coordinates. In version 25 tests on CO2 and acetylene it needed 6 steps, against 12 and 24 in internal coordinates.

One experienced user found Priroda's optimizer weaker than Gaussian's, while its SCF is stronger (2013).

6.3.2 Crash with `process 0 got signal 6` and `too few`

Symptom. The optimization dies with `process 0 got signal 6`. In the output of the last step, the `INTERNAL COORDINATES` table ends with `too few`.

Fix. Optimize in Cartesian coordinates:

```
$optimize cartesian=1 $end
```

This was the answer from one experienced user; the asker did not report back.

The crash could not be reproduced in version 25 tests with molecules that become linear, the usual suspects for running short of internal coordinates. CO2 and acetylene started bent converged in internal coordinates: the optimizer rebuilt its coordinate set when the molecule became linear (from 5 to 4 coordinates for CO2, from 9 to 7 for acetylene) and carried on. They needed 12 and 24 steps, against 6 each with `cartesian=1`.

6.3.3 The optimization "converges" in one step

A run that reports convergence after a single step, with an energy that is not a number, did not optimize anything: the SCF failed on the first step. See *NaN energy, or an optimization that stops after one step*.

6.3.4 A symmetric structure stays symmetric

Without the symmetry options, Priroda works in C1, but a symmetric starting geometry often stays symmetric through an optimization that converges normally, and the Hessian then shows a saddle point. If the optimization did not break the symmetry, always compute the Hessian. If it has an imaginary frequency, paste its `$Energy` block into the input, displace the geometry along that mode if needed, and optimize again.

Version 25 can also use symmetry on purpose: `$molecule sym=1,0.01` detects the point group and symmetrizes the input geometry, and `$optimize sym=1` keeps that symmetry during the optimization. Both options are needed. See *$molecule sym* and *$optimize symmetry*.

6.3.5 A scan starts from a mangled geometry

Cause. Priroda's dihedral angles have the opposite sign to the convention used by Molden, MOPAC and GAMESS. A dihedral *value* copied from another program makes the first scan point a scrambled structure that has nothing to do with the input geometry.

Fix. Flip the sign of every dihedral value taken from another program. The user who hit this confirmed that flipping the signs fixed it. The user who answered (amge) had reported the sign difference on qchem.ru in 2008, for "all versions" of that time. Version 25 still uses the opposite sign: a Z-matrix dihedral of $+100^\circ$ gave Cartesian coordinates with a standard (IUPAC) dihedral of -100°, and a scan whose first `value` was -100° instead of $+100^\circ$ started by turning the molecule into its mirror image.

A scan that requested 10 points but computed only 7 (and 11 of 20) was reported once and never explained.

6.3.6 IRC gives only two points, or no curve

An IRC must start from a properly optimized transition state with an accurate Hessian in the input. Find the TS first (`task=optimize` with *saddle=1*), compute the Hessian there, check that it has exactly one imaginary frequency and that this mode is the reaction you want, and only then run `task=irc`. In one reported case, an IRC started from an unoptimized geometry gave two points with *back=0* and no curve with `back=1`; an experienced user called this typical of an under-optimized geometry.

6.3.7 IRC stops with `Vicinity of a Stationary Point(?) --- STOP.`

This message is also the normal end of an IRC that has reached a minimum: a version 25 test IRC of the ammonia inversion ended with it after 7 points, at the minimum. It is a problem only when it comes early.

On the first IRC step this means that the energy did not go down on that step. The reported fix is to change the step size; see *radius*:

- on a very flat barrier top, increase the step so that it reaches the slope;
- on a very narrow barrier, such as a proton transfer, decrease it, because a step suited to heavy atoms throws the proton too far.

This advice comes from one experienced user.

In a version 25 test on a hydrogen shift (HCN $\rightleftarrows$ HNC), step sizes of 0.03 and 0.1 (the default) reached both minima without stopping early; 0.3, run toward HCN only, also reached its minimum. So the default is a sensible start even for a light atom. The step size changed the cost and how close to the minimum the path ended: 60–69 points with 0.03, 19–21 with 0.1, and 7 with 0.3, which stopped farthest from the minimum (gradient 0.010 against 0.0003–0.006).

An IRC can also end with `Energy Minimum Passed --- STOP.` right after the `Vicinity` message, when its last point overshot the minimum and came out higher in energy than the one before. This is a normal end too.

6.3.8 IRC ends near a minimum but not in it

This is normal: always re-optimize both IRC endpoints. An IRC can also stop on a flat shelf where the gradient is small, or at another TS. Then compute a Hessian at the stopping point and run a new IRC from there; this sometimes has to be repeated three or four times. Priroda has no option to recompute the Hessian every n steps.

6.4 Wrong or suspicious results

6.4.1 Hybrid functional silently ignored

Cause. Hybrid functionals (B3LYP, PBE0/PBE1, ...) are applied only with *theory=ridft*. With `theory=dft` or `theory=tddft`, older versions (reported for v6 to v8) ran without any error message but used zero exact exchange: the exact-exchange part was simply dropped, so the result matched neither the hybrid nor a pure GGA. The developer was told about it and, according to a user who corresponded with him, did not intend to change it.

Newer builds (reported in 2014) refuse with `theory=dft`: the job prints `Basis set input:` twice and stops with `application called MPI_Abort(MPI_COMM_WORLD, 1) - process 0` (on stderr), before the summary box. Another user reproduced it with a known-good input, and version 25 does the same.

Warning

With `theory=tddft` the trap is still open in version 25. A water run with `functional=PBE1` showed `PBE1` in the summary box, finished normally and printed excited states, but the exchange energy was −6.61 instead of about −8.95 hartree and the total energy was about 2.2 hartree too high: the exact-exchange part was simply left out. Any TDDFT result with a hybrid functional is wrong.

Fix. Use `theory=ridft` for every hybrid ground-state calculation, and a pure GGA functional (PBE, the default) for TDDFT. There is no hybrid TDDFT in Priroda.

6.4.2 NaN energy, or an optimization that stops after one step

Cause. The SCF energy on the first step is not a number, so the optimizer stops at once and reports convergence. The reported causes are all basis problems:

- **No basis for an element.** A molecule containing Bi was run with `3z.bas`, which covers only Z ≤ 54. Use `sbk.bas` (ECP basis up to Rn, Z ≤ 86) or `basis4.in` with *four=1*.
- **ECP not read.** In one case nothing was computed and there was no error message. The output showed zeros for one element in the basis table, and no sign that the ECP had been read. The cause was a defect in the user's basis file; `20+sbk.bas` worked.
- A NaN at the first SCF iteration for [PtCl6]2- with the St-rsc-97 ECP basis went away after switching to the `sbk(u)` basis. Why that basis failed is not known.

What to check. In the output, the ECP should be read before the basis, and every atom in the basis table should have a non-zero number of basis functions.

6.4.3 TDDFT oscillator strengths are all zero

A closed-shell TDDFT run on version 25 printed non-zero oscillator strengths in its `Electronic Spectrum:` table with the default settings.

The reported cause of all-zero values: with `restrict=0` set explicitly, singlet and triplet excitations get mixed. Use *$scf restrict=1* (the default for a closed-shell singlet) and compute one multiplicity per run. The user who reported the problem confirmed the fix.

6.4.4 The dipole moment is zero

Version 6 printed `dipole = 0.000000 au ( 0.000000 D)` for every molecule when relativistic basis sets were used, even with `task=dipole`. Version 8 and later print a value.

6.4.5 Energies differ between Priroda versions

Newer builds are faster but give slightly different total energies for the same input. Do not compare close energies computed with different versions; within one version the comparison is fine.

6.4.6 The printed energy components do not add up to E

The line `T = ... V = ... J = ... Ex = ... Ec = ...` does not include the nuclear repulsion energy, so the components do not sum to `E`. This is not an error: E = T + V + J + Ex + Ec + E(nn). The answer came from the developer, relayed by a user.

6.4.7 Thermochemistry of single atoms

For a monoatomic species the enthalpy has an extra RT, and the rotational entropy is printed as `-inf`. Correct these by hand and keep only the translational terms.

6.4.8 Results depend on the number of processes

In one unstable TS search started from a poor guess, the optimization followed different paths with different `-np`. Normally the results differ only in insignificant digits.

6.5 Memory, disk and scratch errors

All *$system* limits apply per MPI process. With `disk=10` on four processes, up to 40 GB can be used.

6.5.1 `need ... bytes.`

Cause. The *memory* limit in `$system` is too small for the job. Version 25 stops right after the basis-set echo with a line such as `need 12544 + 7905408 = 7917952 bytes.` (here for water with `mem=2`); the last number, in bytes (here about 7.6 MB), is what the job needs at that point. Set `memory` (in MB) above it.

6.5.2 `malloc(...) failed`

Cause. Priroda could not allocate the memory requested in *memory*, plus any RAM disk requested with a negative *disk*. The number in the message is the request in bytes.

Fix. Ask for less than the node really has free:

- With a RAM disk, `memory` plus —`disk`— (both in MB), times the number of processes, must fit in physical RAM. One experienced user's example for a 24 GB node, `memory=9000 disk=-14000`, assumes one process per node. Another user's limit, `memory=9250 disk=-9250`, fits this rule.
- The 32-bit Windows (Cygwin) builds cannot allocate more than about 1 GB. 512 MB was enough for most DFT jobs. Several users confirmed this limit. The 64-bit Windows build `p1.exe` ran with `mem=3000`.

A message beginning `We need 1.5 GB of memory...` was reported once. One reply guessed that it is about RAM (`memory=`), not disk space. It may be an older form of the `need ... bytes.` message above.

6.5.3 `out of disk space`

Symptom. `0: out of disk space (VNNNN), <N>KB`, or a similar message naming another array, such as `(ci.Rnvvo)`. Version 25 with a 1 MB RAM disk (`disk=-1`) printed `0: out of disk space (grid_W), 8213824`.

Fix.

- Raise *disk* above the amount reported. Remember that the value is per process.
- Conventional MP2 is very disk-hungry. Use `theory=rimp2` instead.
- Check that `cartesian` or `z-matrix` in `$molecule` matches the coordinates you actually gave; wrong coordinates can make the job far bigger than intended.

Each scratch file is created for 1 GB per process and filled as needed. Jobs needing more than 20–45 GB of scratch were, in one experienced user's words, not worth their cost.

A failure to read a scratch file during MP2 (`0: read file ./tmp/...`) was given the same fix.

6.5.4 A small RI-MP2 job does not fit in memory

A 7-atom RI-MP2 job with `four=1`, `basis4.in` and `set=L22` claimed not to fit in memory however much was given, while smaller and larger basis sets ran. A second user reproduced it and got `0: write VMNN(11753704:14042536)?`. The first user found that direct integrals avoid the problem:

```
$integral direct=1 $end
```

See *direct*.

6.5.5 Where scratch files go

Priroda writes scratch files to the directory given by *path*; if `path` is not set, to `$TMPDIR`; otherwise, in version 25, to `/tmp` (a 2020 user report said the directory of the input file). On a cluster, `/tmp` is often too small or quota-limited for large scratch files. Use the cluster's local scratch area, or a RAM disk with a negative `disk` value. Never put scratch on NFS.

A job that stops while writing a scratch file, with a message like `0: write file /tmp/tmp9633.11 (leAX): fd=21 ...`, was reported once (2020) during an analytic Hessian on a cluster. An experienced user replied that no `path=` was set, so scratch went to `/tmp`, probably on the master node, where quotas can make the job fail, and advised the cluster's scratch area. The asker then tried a `tmp` folder and got the same error; the case was not resolved. Check the free space, the quotas and the `disk=` setting.

If Priroda does not get enough memory or scratch, it stops with an error; it does not fall back to a slower algorithm.

6.5.6 Slow runs: disk speed

Priroda is very demanding of disk speed. For all methods except GGA DFT, `input.txt` warns that performance degrades "terribly" if more than one process reads and writes large files on the same physical drive. Options, in order of preference:

- keep scratch in RAM with a negative *disk* value (one cluster reported a 6× speed-up for RI-MP2);
- give each process its own physical drive with a colon-separated *path* list;
- if the job is disk-bound, switch to direct integrals with *direct=1*. This costs CPU time but can be much faster overall: one RI-MP2 optimization went from 2 to 20 steps per night.

6.6 Parallel and MPI problems

6.6.1 A parallel RI job hangs with no message

Cause. RI methods, RI-MP2 and RI-DFT included, hang forever if any process gets no work. This happens when the job is too small for the number of processes. Two experienced users confirmed it and called it a bug.

How to recognize it. After `integral estimates`, the output lists the blocks per node. In a normal run every node has work, as here; a hung job shows `0 blocks` on some node:

```
   2892490 blocks,    103625770 integrals on node  0
   3681560 blocks,     91256706 integrals on node  1
```

Fix. Run small RI jobs on fewer processes, or on one core.

Older reports describe similar hangs: small jobs on many processors hang (v6/v7); MP2 on a 102-atom complex died with signal 8 on more than about 24 cores (v7, MVAPICH); a v6 Hessian on 4 cores could hang at `Solving for responses ...`. The v6 advice was at most 2 cores for a Hessian and at most 4 for an optimization. Another user ran DFT, including Hessians, on 24–32 cores without trouble, so this advice is disputed. In 2018 an experienced user said the Windows builds run in parallel only from version 16, and that the v6 Hessian uses at most 2 cores, the v17 one 4 or more.

6.6.2 Little or no speed-up from more cores

- Reports on DFT scaling within one node disagree: one experienced user found at most 10% more from 32 cores than from 6–8, while another measured 101, 63 and 37 s for the same optimization on 8, 12 and 24 cores.

- For DFT, running across several nodes gives no gain; one experienced user said so on three occasions. RI-MP2 is different: in a v8 benchmark on 4 nodes with scratch in RAM it ran 6× faster than on 2 cores of one node.
- In a DFT run most of the memory and all the disk I/O stay on rank 0, so spreading a DFT job over nodes does not spread its memory. RI-MP2 distributes memory better.
- For throughput, several single-core jobs at once can beat one multi-core job when the jobs are not disk-bound.
- For RI-MP2, the v5 documentation suggests a number of processes that divides the number of occupied orbitals.

6.6.3 `Environment is too large (max is 193)`

Symptom. A parallel run started with the `mpiexec` from the official site prints this line (once per process) and stops at once; no output file is written. Typically this happens inside a batch job.

Cause. That `mpiexec` refuses to start when the environment holds 193 or more variables. Batch systems add many variables of their own; a Slurm job in a version 25 test had 210.

Fix. Start it with a minimal environment:

```
env -i PATH="$PATH" HOME="$HOME" TMPDIR="$TMPDIR" ./mpiexec -np 4 ./p input.in output.out
```

See *Batch systems* for a complete job script.

6.6.4 Starting a parallel run on Linux

A parallel version 25 run must be started through an MPI launcher, such as the `mpiexec` from the official site: `./mpiexec -np 4 ./p job.in job.out`. `./p -np 4 job.in job.out`, which worked in versions 6 and 10, now stops with `input file '-np'?.`. With version 10, starting through the bundled `mpiexec` once failed with `Error in system call mpiexec: No such file or directory` or `mpiexec could not exec p`. Use absolute paths for the binary, the input, the output and the basis file.

The message `OOPS: semop lock failed` (with `process N received signal 2` or `6`) has two reported contexts:

- as a secondary message after another fatal error, such as running out of disk. Fix the first error;
- in v6 shared-memory runs on 7 cores. Fewer cores avoided it; cleaning semaphores with `ipcrm` did not help.

It also appeared for one user who ran as a normal user while the same input worked as root. That case was never resolved.

A run that stops producing output right after the header, while the process keeps using CPU, has been reported and called common. The only advice was to look for stray files named after the input or output in the input, output, program and home directories, and then to rule out causes one by one.

6.6.5 Windows

Old Windows builds were serial only. For a parallel 64-bit run (version 16 or later), users report this setup:

1. From a 64-bit Cygwin installation with OpenMPI, copy `cygwin1.dll`, `cyggcc_s-seh-1.dll`, `cygmpi-12.dll`, `cygopen-pal-13.dll`, `cygopen-rte-12.dll` and `orterun.exe` next to the Priroda executable. Take `cygwin1.dll` from the same Cygwin installation, because the other DLLs work only with it. These are the names from a 2017 Cygwin. A 2018 report says newer OpenMPI packages ship `cygopen-pal-40.dll` and `cygopen-rte-40.dll` instead. Those do **not** work with `p_190306.exe`, even renamed: see the note below.

2. To run outside the Cygwin terminal, create a `dev` directory one level above the directory that holds `cygwin1.dll`, with subdirectories `shm` and `mqueue`. For Priroda in `C:\Priroda 16` that means `C:\dev\shm` and `C:\dev\mqueue`. Without them Cygwin prints `Warning: '/dev/shm' does not exists or is not a directory.`

3. Set `TMPDIR` and start the run from `cmd.exe`. The command as posted was:

```
cd /d "C:\Priroda 16"
set TMPDIR="%TEMP%"
"orterun.exe" -np 2 ./p16_64_parallel.exe "C:\Work\file.in" "C:\Work\file.out"
```

   Write `set "TMPDIR=%TEMP%"` instead: with `set TMPDIR="%TEMP%"` the quotes stay in the value, and the run stops at `0: opening file ...` unless the input sets `path=`.

4. If the program is in a system directory such as `C:\Program Files`, run it as administrator.

Note

A test in 2026 on Windows 10 (with a Cygwin installation on `PATH`) got `p_190306.exe` running on 2 processes. It needed the `-12`/`-13` DLLs and `orterun.exe` from an older OpenMPI package (files dated 2016-06-21). With the current `-40` DLLs, or those renamed to the old names, the programs exited at once with code -1073741515 (0xC0000135, a required DLL was not found). The path of the Priroda executable had to be written with forward slashes: `orterun.exe -np 2 C:/Priroda/p_190306.exe file.in file.out`. With backslashes, `orterun.exe` stopped with `orterun:exe-not-found`. The full setup is in *Parallel runs on Windows*.

Network problems reported with this setup:

- On a PC connected by WiFi that also has a LAN adapter, OpenMPI failed with `tcp_peer_send_blocking: send() to socket 12 failed`. Disabling the LAN adapter fixed it (single report).
- With the Ethernet cable unplugged, only some cores were used. Plugging it back in fixed it (single report).

A 2021 attempt to follow this recipe on Windows 10 failed with an error that was posted only as a screenshot; no fix was posted. The `-40` DLLs described in the note above may have been the cause. One user runs Priroda under WSL on Windows instead; no details of that setup were posted.

Warning

Not every build on the official site is reliable. A note there once said that the `w64/*.exe` 64-bit Cygwin builds had a DFT/ECP bug. If a Windows build misbehaves, ask the developer which build you should be running.

CHAPTER

SEVEN

KEYWORDS IN THE PROGRAM

This page lists the keyword names that the Priroda 25 (2025.02.28) executable reads, found by examining the program file itself. Names that this manual documents are links into the *input reference*; the others are only known to exist. Their meaning is unknown, so do not use them in production work.

A name listed here is accepted by the program; a name that is not listed stops the run with `*** error in '$group' : unknown name ...`, as long as the task reads that group. Any unique prefix of a name is accepted, so `sym=1` in `$optimize` means `symmetry=1` and `dist=` in `$uniq` means `distance=`.

7.1 Groups for everyday work

Group	Keywords
$system	*memory*, *path*, *disk*
$control	*task*, *theory*, *four*, a137, *state*, weight, *basis*, *nucleus*, field, *derivative*, fit, wavelength, *save*, *print*
$scf	*restrict*, *convergence*, *iterations*, *procedure*, *d1small*, *core*, occupation, fraction, temperature, ttype, diagm, xmax, n1, fitxc, eemin, mab, aw, wv
$dft	*functional*, zero, n, p
$grid	*accuracy*, type, cutoff, nradial, nadd, use43
$integral	*direct*, *routine*, storage, invert, cutij, vcut
$ator	*core*, *mcore*, nc, no, active, mocc, occupation, diagonalize
$gues	*read*, *mix*, *na*, *nb*, n, a, type, gradient, localize
$optimize	*saddle*, *tolerance*, *trust*, *steps*, *follow*, *points*, *back*, down, *radius*, *fix*, *value*, *coordinates*, *cartesian*, *symmetry*, *gradient*, *print*, exact, bend, local, circle, weight, massweight, minimize, maximize, cubicfit, energy, de0, fragment, noise, state
$d2ec	*length*, *steps*, *displace*
$ther	*sigma*, *temperature*, *mass*, project, localize, diagonalize
$nmr	*standard*, *points*
$Energy	*e*, *g*, *h*, *d*, *a*, *dx*, *ax*, r
$uniq	*distance*, file
$dynami	t, *e*, *n*, *m*, a, b, c, *i*, o, z, y, *dx*, kd, lw, *hb*, rb, pb, *v1*, frag, *file*, fprint

`$molecule` is read differently from the other groups, so its keywords (`charge`, `mult`, `cartesian`, `z-matrix`, `set`, `unit`, `sym`) could not be listed from the program; see *$molecule*. Group names are case-insensitive: the

program looks for `$energy`, which is the same group as `$Energy`. It is also read as `$energy001`, `$energy002` and so on, for tasks with several structures.

7.2 Groups of little-documented tasks

These groups belong to tasks and methods that no source describes in detail. They are listed only so that a name met in an example input can be recognized.

Group	Keywords	Task or method
`$mcscf`	`core`, `occupied`, `virtual`, `excitation`, `program`, `convergence`, `iterations`, `diagonalize`	MCSCF methods
`$ci`	`iterations`, `ndiis`, `ecut`, `v4`	CI methods
`$espmap`	`d`, `n`	electrostatic-potential map
`$optimize` (second reader)	`function`, *model*, `h0`, `delta`, `fragment`, `steps`, `trust`, `tolerance`, `print`, `fix`, `value`	`task=opt_n1`
`$path`	`distance`, `points`, `spring`, `steps`, `trust`, `tolerance`, `tension`	`task=path`, `path0`
`$chain`	`points`	`task=chain`
`$dynamic` (other readers)	`t`, `e`, `dt`, `box`, `print`; `t`, `e`, `dt`; `t`, `e`, `dt`, `v`, `r`, `nt`, `nit`, `file`, `print`	`task=dynamic`, `dynamicn2`, `dynamic1p`
`$walk`	`n`, `m`, `e`, `i`, `b`, `dx`, `lw`, `hb`, `v1`, `fp`	`task=walk`
`$orient`, `$mol1`, `$mol2`	orient: `r`, `lz`, `nr`, `e`, `d0`, `amax`, `set`, `list`, `path`, `file`, `text`, `print`; mol1, mol2: `e`, `z`, `x`, `q`, `m`, `n`	`task=orient`
`$press`	`nadd`, `radius`, `steps`, `trust`, `tolerance`, `sign`, `energy`, `vector`	`task=press`
`$coord`	`index`	`task=forceconst`
`$steric`	`g`, `v`, `w`, `n`, `r`, `e`, `i`	`task=steric`
`$dimer`	`job`, `order`, `distance`, `radius`, `step`, `trust`, `gradient`, `energy`, `iterations`, `tolerance`, `delta`, `analytic`, `file`, `print`	`task=dimer`
`$anchor`, `$confine`	anchor: `u`, `k` and two more; confine: probably `p`, `h`, `o`, `k`, `m` (not certain)	restraints in molecular dynamics
`$r_n`	`r`, `m`, `n`, `d`, `q`, `file`, `list`	not known
`$surface`	`vx`, `x`, `nx`, `mv`, `r`, `h`, `vo`, `nv`	`task=anharmonic`
`$fmatrix`, `$ce`, `$ca`, `$cb`	fmatrix: `n`, `f`, `s`, `fa`, `fb`; ce: `n`, `c`, `e`, `ca`, `cb`, `ea`, `eb`; ca, cb: `n`, `c`, `e`	starting orbitals given in the input
`$scf` (QM model)	`restrict`, `convergence`, `iterations`, `direct`, `print`, `mc`, `amix`, `ijmix`, `cut`, `nf`, `nmax`, `nb`, `bond`, `penalty`, `distance`, `option`	`theory=qm_n*`
`$integral` (AM model)	`tolerance`, `atom`, `angular`, `radial`, `distance`, `term`	`task=am_integral`
`$isogrid`	`d`, `s`, `c`, `r`, `m`, `n`, `e`, `p`, `iter`	`task=isogrid`

Further groups (`$molbf`, `$moldf`, `$mbf`, `$mdf`, `$esp`, `$bond`, `$e2r6`, `$geometry`, `$dmtype`, `$mm`, `$order`, `$bf`, `$c6p`, `$hferror` (`angular`, `radial`), `$mpitest`, `$multiply`, `$gtest`) serve the parametrization of the model Hamiltonians or the developer's own tests.

7.3 Task names

The program recognizes these values of *task*. Only the first group is documented or tested.

- Documented: `energy`, `gradient`, `hessian`, `optimize`, `optimize+hessian`, `irc`, `scan`, `nmr`, `dipole`, `anharmonic`, `thermostat`, `uniq`, `opt_n1`, `saddle`, `transit`, `extremal`, `conical`, `path`.

- Not documented: `irc4`, `dipole1`, `dipole1x`, `dipole2`, `polarizability`, `numeric`, `loop`, `stretch`, `extremal2`, `path0`, `pathr`, `pathx`, `chain`, `press`, `forceconst`, `potential`, `dynamic`, `dynamicn2`, `dynamic1p`, `walk`, `mm0dimer`, `dimer`, `orient`, `guess`, `fragment_guess`, `nss_num`, `steric`, `compare`, `isogrid`, `functional`, `d1fit`, `df1_n4`, `df2_n4`, `df2_e_n4`, `mbf2`, `molbf`, `moldf`, `dmtype`, `am_integral`, and developer tests (`mpitest`, `multiply`, `etest`, `fmtest`, `ritest`, `riopt`, `test_v`).

`check` is not a task name. Like any unknown task name, `task=check` sets up the run and stops quietly, which is why it seems to work. An unknown `theory=` name is worse: the program runs Hartree–Fock without a message.

7.4 Theory names

Besides the methods described in *theory*, the program contains these method names, none of which is documented: `OSDFT`, `DFT_CNO`, `LEDFT`, `LE1DFT`, `DFCIS`, `RIHFDF`, `MP4D`, `PT4DQ`, `PT4SDQ`, `PT4SDTQ`, `CISD`, `CCD4S`, `CCD4ST`, `CCD&MP2`, `LCCD+MP2`, `SN_CCD&MP2`, `SN_CCSD&MP2`, `LEHF`, `LEHF_N3`, `LEMP2_N3`, `LE2HF_N3`, `LE2MP2_N3`, `MCSCF`, `CASSCF`, `GSCI`, the model Hamiltonians `QM_N2`, `QM_N2D`, `QM_N2B`, `QM_N2B3`, `QM_N2C`, `QM_N2C0`, `QM_N2F`, `AM_N1`, `MM0_N2`, `DM_1`, `BM_1`, `MM_1`, and `6-12`, `FREP`, `SCRIPT`.

CHAPTER

EIGHT

SOURCES

This manual is not an official Priroda document. It was not written or reviewed by the developer of Priroda, Dimitri Laikov. It is compiled from the sources below, which are listed roughly in the order of how much weight the manual gives them when they disagree. Each keyword entry in the *input reference* names the sources it rests on.

8.1 Test runs

Version 25 test runs
: Small test inputs run on Priroda 25 (2025.02.28) in October 2026 for this manual, first on the project owner's Linux machine and in later rounds also on a 4-core Linux virtual machine. Most use water, ammonia, OH or O_2; others use CO_2, acetylene, HCN, H_2, Cr_2 and a 22-atom Cu(II) complex. Where they confirm or contradict an older source, the manual follows the run and says so. The inputs and outputs are kept with the manual's source files.

Version 25 executable
: The Linux build `p4_d` from the official site, whose banner matches the test runs, was examined without running it to list every group, keyword, task and theory name it reads; the same build was later used for the test runs on the virtual machine. This list settles which names exist; it says nothing about what they do. The result is on *Keywords in the program*.

8.2 Program files from the developer

Official file index
: https://rad.chem.msu.ru/~laikov/p_w/, read in October 2026 (page dated June 2025). A bare listing of executables, example inputs and outputs, basis-set and parameter files, and the helper scripts `ge`, `gm` and `grx`. It has no prose documentation. Example outputs from this site (versions 13–15) are the strongest evidence for those versions; for version 25, the test runs take precedence. Windows builds are in the `exe64/` subdirectory.

8.3 Unofficial keyword lists

None of these documents is official, and none comes from the developer's site. The three files `input.txt`, `Dft.rtf` and `Dft_ansi.txt` were provided by Alexandra Freidzon.

Input Summary (`input.txt`)
: An English keyword list for a recent version, with user annotations appended. It is the most complete keyword list available, but it is not an official document. A few of its Russian annotations are corrupted and cannot be read.

Version 5 keyword reference (KNC wiki)
: A keyword reference for version 5 on the wiki of the Kazan Scientific Centre computing centre, next to a beginner FAQ. Who wrote it is not known. The wiki is offline; pages were recovered from the Internet Archive (captures 2008–2012). A PDF copy of the documentation page is at https://monitor.hpc.umanitoba.ca/doc/Priroda_Documentation_from_KNCWiki.pdf.

`Dft.rtf`
: A short Russian keyword list with practical advice, from an older version. Some of its defaults differ from later versions.

`Dft_ansi.txt`
: A Russian description of the input of Priroda's predecessor. Its group names are obsolete; it is used here only for history.

8.4 User-written material

Forum statements attributed to the developer reach these sources only second-hand, through users who relayed his e-mails. The manual marks such statements as relayed.

chemport.ru forum, thread 28803 ("Priroda")
: https://chemport.ru/forum/viewtopic.php?f=71&t=28803. The main Russian-language user thread, 518 posts from 2009 to 2026. The largest single source for practical advice and known problems.

chemport.ru forum, thread 112860 ("Difficult cases with Priroda")
: https://chemport.ru/forum/viewtopic.php?f=71&t=112860. Six posts (2015–2016) on SCF divergence, RI-MP2 memory and parallel hangs, with complete inputs and outputs.

qchem.ru forum, thread "Quantum-chemical program Priroda"
: The forum of the Ufa Quantum Chemistry Society, 2007–2012. The site has been offline since about 2019; posts up to August 2012 were recovered from the Internet Archive.

LIMOR page, Novosibirsk Institute of Organic Chemistry
: http://limor1.nioch.nsc.ru/priroda.html. Example inputs, tables of basis-set names, and Perl helper scripts (`pri`, `priin`, `pri2mol` and others) written by users. Much of what the manual says about the output format was first inferred from how these scripts parse it and has since been checked against version 25 output.

8.5 Published papers

The papers listed under *How to cite* describe the methods behind Priroda. They are cited where the manual explains a method, not as documentation of the input.

INDEX

Symbols

$Energy, 61
$atoms, 52
$control, 36
$d2edr2, 58
$dft, 49
$dynamic, 63
$grid, 50
$guess, 45
$integral, 51
$molecule, 43
$nmr, 60
$optimize, 52
$scf, 47
$system, 34
$thermo, 59
$uniq, 65

A

A (*$Energy*), 62
accuracy (*$grid*), 50
Ax (*$Energy*), 63

B

back (*$optimize*), 55
basis (*$control*), 40

C

cartesian (*$optimize*), 54
charge (*$molecule*), 43
convergence (*$scf*), 47
coordinates (*$optimize*), 56
core (*$atoms*), 52
core (*$scf*), 49

D

D (*$Energy*), 62
d1small (*$scf*), 49
derivative (*$control*), 42
direct (*$integral*), 51
disk (*$system*), 35
displace (*$d2edr2*), 58
distance (*$uniq*), 65
dx (*$dynamic*), 63
Dx (*$Energy*), 62

E

e (*$dynamic*), 63
E (*$Energy*), 61

F

file (*$dynamic*), 64
fix (*$optimize*), 56
follow (*$optimize*), 54
four (*$control*), 40
functional (*$dft*), 49

G

G (*$Energy*), 62
gradient (*$optimize*), 57

H

H (*$Energy*), 62
hb (*$dynamic*), 64

I

i (*$dynamic*), 64
iterations (*$scf*), 48

L

length (*$d2edr2*), 58

M

m (*$dynamic*), 64
mass (*$molecule*), 44
mass (*$thermo*), 60
mcore (*$atoms*), 52
memory (*$system*), 34
mix (*$guess*), 46
model (*$optimize*), 58
mult (*$molecule*), 43

N

`n` (*$dynamic*), 64
`na` (*$guess*), 45
`nb` (*$guess*), 46
`nucleus` (*$control*), 41

P

`path` (*$system*), 35
`points` (*$nmr*), 61
`points` (*$optimize*), 55
`print` (*$control*), 41
`print` (*$optimize*), 57
`procedure` (*$scf*), 48

R

`radius` (*$optimize*), 55
`read` (*$guess*), 45
`restrict` (*$scf*), 47
`routine` (*$integral*), 51

S

`saddle` (*$optimize*), 53
`save` (*$control*), 42
`set` (*$molecule*), 44
`sigma` (*$thermo*), 59
`standard` (*$nmr*), 60
`state` (*$control*), 40
`steps` (*$d2edr2*), 59
`steps` (*$optimize*), 54
`sym` (*$molecule*), 45
`symmetry` (*$optimize*), 57

T

`task` (*$control*), 36
`temperature` (*$thermo*), 60
`theory` (*$control*), 38
`tolerance` (*$optimize*), 53
`trust` (*$optimize*), 53

U

`unit` (*$molecule*), 44

V

`v1` (*$dynamic*), 64
`value` (*$optimize*), 56